学校代号 10532　　学　　号 S1910W0757

分 类 号 TP391　　密　　级



湖南大学
HUNAN UNIVERSITY

工程硕士学位论文

CPU 和 GPU 上的稀疏张量 MTTKRP 的并行算法设计和优化

学位申请人姓名 胡蓉

培 养 单 位 信息科学与工程学院

导师姓名及职称 周旭 副教授

杨志邦 高级工程师

学 科 专 业 计算机技术

研 究 方 向 并行计算

论 文 提 交 日 期 2021 年 4 月 30 日

中国知网 https://www.cnki.net

学校代号：10532

学号：S1910W0757

密级：

湖南大学工程硕士学位论文

CPU 和 GPU 上的稀疏张量 MTTKRP 的并行算法设计和优化

学位申请人姓名：胡蓉

导师姓名及职称：周旭 副教授

杨志邦 高级工程师

培 养 单 位：信息科学与工程学院

专 业 名 称：计算机技术

论文提交日期：2021 年 4 月 30 日

论文答辩日期：2021 年 5 月 21 日

答辩委员会主席：陈浩 教授

中国知网 https://www.cnki.net

Design and Optimization of Parallel for MTTKRP of Sparse Tensor on CPU and GPU

by

HU Rong

B.E. (Chang'an University) 2019

A thesis submitted in partial satisfaction of the

Requirements for the degree of

Master of Engineering

in

Computer Technology

in the

Graduate School

of

Hunan University

Supervisor

Associate Professor Zhou Xu

Senior Engineer Yang Zhibang

May, 2021

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

摘　要

随着信息获取技术的发展，现实数据呈现多维度高稀疏的发展趋势。传统的矩阵分解这种把高维数据转化成矩阵的处理方式已经无法满足大数据时代的要求。稀疏张量出现在许多具有多维和稀疏数据的大规模应用程序中，张量分解是矩阵分解在高维空间上的扩展。不同于矩阵分解，张量分解在提取数据内在的低维结构特征的同时，保留数据在高维空间的隐藏联系。矩阵化张量乘 Khatri-Rao 乘 (MTTKRP) 是张量分解中的基本运算，尤其是在 CANDECOMP/PARAFAC (CP) 分解中。CP 分解将一个张量分解为若干性质更简单的张量，常用交替最小二乘法 (ALS) 进行并行加速。在求解大型稀疏张量的 ALS 算法中，其主要计算量是 MTTKRP，因此高效计算 MTTKRP 是提升张量分解的核心重要环节。近年来 GPU 的快速发展，其独特的并行架构适合密集型和高度并行的计算。然而不规则的计算模式和稀疏结构以及稀疏张量操作的大内存占用使得稀疏张量计算具有很大的挑战性。

本文从加速张量运算中的 MTTKRP 计算着手，以并行技术为基础，首先针对现实数据的高维稀疏特点，采取压缩格式进行存储提高内存利用率；然后针对不同压缩格式，设计 CPU 并行实现算法；最后结合 GPU 的并行方式和存储特点，设计 GPU 的 MTTKRP 并行算法，优化线程执行方式，提升算法的并行效率。

本文的主要工作如下：

针对张量的高维稀疏特性，分析了稀疏张量的不同存储格式。在 CPU 平台上设计不同稀疏格式的 MTTKRP 并行算法，减少对稀疏张量中的空切面和空纤维的计算。考虑到纤维和切面的索引与因子矩阵的关联，本文将计算过程分解为向量内积操作，即纤维与矩阵列计算，减少 MTTKRP 的迭代次数和冗余计算。实验证明，在 CPU 上针对电影标签数据集最高可以实现 3.67 的加速比。

基于分层的张量存储格式设计 GPU 的并行算法。首先将稀疏张量按照切面进行分组，提出切面级并行的 MTTKRP。其次，提出合并存储的 MTTKRP，实现合并访存和数据复用，实现纤维之间的并行。之后使用共享内存存储切面的中间结果，降低访存延迟。最后设计线程束执行方式以提高算法并行度，采用交错归约按行得到结果矩阵。对于本文提出的张量运算中的 MTTKRP 优化算法，使用公开电影标签数据集测试。实验证明，在 GPU 上平均实现 1.59 倍加速，最高可达 3.58 倍的加速。

关键词：MTTKRP；并行计算；稀疏张量；张量分解

中国知网 https://www.cnki.net

Abstract

With the development of information acquisition technology, the real data presents a development trend of multi-dimensional high sparseness. Traditional matrix decomposition, which transforms high-dimensional data into matrix, can not meet the requirements of big data era. Sparse tensors appear in many large-scale applications with multi-dimensional and sparse data. Tensor decomposition is an extension of matrix decomposition in high-dimensional spaces. Different from matrix decomposition, tensor decomposition extracts the inherent low-dimensional structural features of the data while retaining the hidden connection of the data in the high-dimensional space. Matricized-Tensor Times Khatri-Rao Product (MTTKRP) is the basic operation in tensor decomposition, especially in CANDECOMP/PARAFAC (CP) decomposition. CP decomposition decomposes a tensor into several simpler tensors with simpler properties, which are usually accelerated by Alternating least squares (ALS). In the ALS algorithm for solving large sparse tensors, the main calculation amount is the MTTKRP, so efficient calculation of MTTKRP is the core and important link to improve tensor decomposition. With the rapid development of GPU in recent years, its unique parallel architecture is suitable for intensive and highly parallel computing. However, the irregular calculation mode and sparse structure, as well as the large memory footprint of sparse tensor operations, make sparse tensor calculations very challenging.

This article starts with MTTKRP in accelerated tensor operations, based on parallel technology. First, aiming at the high-dimensional sparseness of real data, compressed formats are adopted for storage to improve memory utilization. Then design the CPU parallel implementation algorithm for different compression formats. Finally, combined with the parallel mode and storage characteristics of GPU, the MTTKRP parallel algorithm of GPU is designed to optimize the thread execution mode and improve the parallel efficiency of the algorithm.

The main work of this paper is as follows:

Aiming at the high-dimensional sparsity of tensors, different compressed formats of sparse tensors are analyzed. The parallel algorithm of MTTKRP with different sparse formats is designed on the CPU platform to reduce the calculation of the empty section and the empty fiber in the sparse tensor. Considering the relationship between the index of fibers and slices and the factor matrix, this paper decomposes the calculation process into a vector inner product operation, that is, the calculation of

中国知网 https://www.cnki.net

fiber and matrix columns, reducing the number of iterations and redundant calculations of MTTKRP. Experiments have proved that the optimal speedup of 3.67× can be achieved on the CPU for the Movielens dataset.

Design GPU parallel algorithms based on hierarchical tensor storage format. First, the sparse tensors are grouped according to the slices, and the slice-level parallel MTTKRP is proposed. Secondly, the MTTKRP of coalesced storage is proposed to realize coalesced memory access and data reuse, and realize the parallelism between fibers. After that, shared memory is used to store the intermediate results of the slices to reduce memory access latency. Finally, the warps execution pattern is designed to improve the parallelism of the algorithm, and the result matrix is obtained by row through interleaved reduction. For the MTTKRP optimization algorithm in the tensor operation proposed in this article, the public data set is used to test. These experimental results show that the optimal strategies achieve 1.59× average speedup and achieve up to 3.58× speedup on GPU.

Key Words: MTTKRP; Parallel Computing; Sparse Tensor; Tensor Decomposition

中国知网 https://www.cnki.net

目　　录

学位论文原创性声明和学位论文版权使用授权书 I
摘　要 II
Abstract III
插图索引 VII
附表索引 VIII
第 1 章 绪　论 9
1.1 研究背景及选题意义 9
1.2 国内外研究现状 10
1.2.1 国外研究现状 10
1.2.2 国内研究现状 12
1.3 本文研究内容 12
1.4 论文结构安排 13
第 2 章 相关理论基础 14
2.1 CUDA 模型 14
2.1.1 CUDA 编程模型 14
2.1.2 CUDA 存储模型 16
2.2 张量与张量运算 18
2.2.1 张量的基本概念 18
2.2.2 张量的基础运算 19
2.3 CP 分解算法 21
2.4 本章小结 24
第 3 章 基于 CPU 的 MTTKRP 算法分析 26
3.1 张量的压缩格式 26
3.1.1 COO 格式 26
3.1.2 CSR 格式 26
3.1.3 CSF 格式 27
3.2 COO 格式下 MTTKRP 并行算法分析 27
3.3 CSF 格式下 MTTKRP 并行算法分析 30
3.4 实验 32
3.4.1 实验环境 32
3.4.2 实验结果与分析 33
3.5 本章小结 34
第 4 章 基于 GPU 的 MTTKRP 算法优化 35

中国知网 https://www.cnki.net

4.1 并行算法基本实现 35
4.2 并行算法优化方式 36
4.2.1 切面并行 36
4.2.2 合并访存 37
4.2.3 共享内存 38
4.3 优化并行算法的实现 40
4.4 实验 42
4.4.1 实验环境 42
4.4.2 实验结果与分析 43
4.5 本章小结 46
结论 47
参考文献 49
附录 A 攻读学位期间发表的学术论文与获得的成果 54
附录 B 攻读学位期间参加的科研项目 55
致　谢 56

中国知网 https://www.cnki.net

插图索引

图 2.1 NVDIA GPU 架构 14
图 2.2 kernel 函数的示例.....15
图 2.3 GPU 内存层次.....16
图 2.4 线程顺序访问存储体 17
图 2.5 三维张量的表示.....18
图 2.6 三维张量的纤维.....19
图 2.7 三维张量的切面.....19
图 2.8 三维秩一张量.....22
图 2.9 三维张量的 CP 分解 23
图 3.1 MTTKRP 的运算过程 30
图 3.2 CSF 格式下的算法加速比 34
图 3.3 COO 格式下算法的加速比 34
图 4.1 切面的划分方式.....37
图 4.2 线程块中线程的操作 38
图 4.3 线程束产生的共享矩阵 39
图 4.4 共享矩阵按行归约求和 40
图 4.5 Opt-MTTKRP 和 CSF-MTTKRP 的加速比.....43
图 4.6 基于 ParTI-COO 的加速比.....44
图 4.7 基于 B-CSF 的加速比 45
图 4.8 基于 HB-CSF 的加速比 45
图 4.9 优化策略对性能的影响 46

中国知网 https://www.cnki.net

附表索引

表 3.1 数据集信息 32
表 3.2 CSF 格式下的 MTTKRP 串行算法时间 33
表 3.3 COO 格式下的 MTTKRP 串行算法时间 33

中国知网 https://www.cnki.net

第 1 章 绪 论

1.1 研究背景及选题意义

大规模实际问题需要数学理论和计算机程序进行分工合作，数值算法常使用基于矩阵的线性代数运算来解决一些计算问题。多重线性代数是线性代数的一种发展，它提供了一种在高维空间上进行计算的语言。传统的高维数据分解方法是将数据转化成矩阵的形式，利用奇异值分解[1]等矩阵分解方法提取数据特征。但是这种转化方法会失去数据在高维空间的结构特征，降低特征因子的准确性。矩阵分解无法适应高维空间的复杂关系，因此将其拓展到处理高维数据的张量分解上，直接利用张量分解处理高维数据可以保留数据的潜在关联。

张量作为一种多维数组结构，允许任意数量的维度，其中每个维度称为模态。常用的标量，向量和矩阵都是存在于不同维数中的所有特定类型的张量，都需要不同数量的索引变量来访问数据。不同于矩阵，张量可以保留数据在高维情况下的潜在关联，承载了更多的数据信息。通过推广到更高的维度，张量提供了强大的抽象能力。张量已广泛应用于各个领域，例如图像和视频补全[2-5]，信号处理[6-9]，图像分类[10-13]和人脸识别[14-17]等等。在电影评分上的推荐系统将电影，用户和评论日期与用户评级相关联，以产生新的电影推荐[18]。与评级相关的有三个指标变量，可以产生一个三维的评级张量。用户对电影的选择有限，用户与电影之间的这种稀疏关系导致张量稀疏，其中大多数评分为零。利用高阶张量近似实现评级数据的压缩，分解得到低维的潜在因子空间。随着数据的关联越来越紧密，数据的规模与体量呈现指数级增长，通常只要一个维度出现稀疏就可以使整个张量稀疏，因此张量在多维空间中有明显的稀疏性。稀疏的张量数据具有不规则的计算模式，高维数和高稀疏的三维数据在做张量分解时变得异常复杂与耗时，存储和时间成本呈现出爆炸式增长。

MTTKRP（Matricized-Tensor Times Khatri-Rao Product）是张量运算的计算瓶颈，尤其在张量分解中。张量分解为多维数据提供了一种数据分析的方法，特别是 CP 分解作为矩阵奇异值分解的一般化，提供了数据的低秩近似。张量计算的优点之一是可以将他们转化为矩阵运算，这些矩阵运算已通过 BLAS 接口针对当今的体系结构进行了优化。几乎所有计算 CP 分解的时间都发生在 MTTKRP 操作中，CP 分解的其余计算步骤在 BLAS 库中进行了高度优化，使 MTTKRP 的优化成为许多研究人员关注的重点领域。MTTKRP 算法对应于矩阵和矩阵的乘法，直接使用矩阵的乘法会产生大量多余的操作。Khatri-Rao 乘积作为一个稠密矩阵，

中国知网 https://www.cnki.net

在张量维度很高的情况下，会占用大量的内存。因此优化高维稀疏张量与 Khatri-Rao 乘积是提升 MTTKRP 运算的关键。

NVIDIA GPU 是高度并行的多核处理器，具有很高的内存带宽。CUDA 作为一个通用并行计算平台和编程模型，使得 GPU 的可编程性不断提高。利用 GPU 的运算特点处理数值计算问题是一个重要的研究方向，GPU 中的线程受到硬件管理，可以实现零开销的线程切换[19]。当前线程处于等待数据返回时，线程调度器可以立刻调度处于就绪状态的线程。GPU 用较小的代价获得高性能计算，将其和张量分解结合可以明显降低张量的处理成本。MTTKRP 运算中许多计算任务都符合 GPU 中的单指令多线程 SIMT 的特性，因此基于 GPU 的并行方式为优化 MTTKRP 提供了可行的解决方案。

综上所述，本文从加速张量运算中的 MTTKRP 计算着手，以并行技术为基础，设计压缩张量格式下 CPU 和 GPU 的 MTTKRP 优化算法。

1.2 国内外研究现状

1.2.1 国外研究现状

稀疏张量计算特别是张量分解在具有多维关系的大量数据中越来越流行。因此，具有高效和可扩展的稀疏 MTTKRP 实现对于各种应用程序是非常有价值的。张量分解是 Hitchcock[20]于 1927 年首次提出，在 Tucker 等人[21]的工作之前，张量的概念只出现在心理计量学中。Appellof 和 Davidson[22]被认为是第一个在化学计量学上使用张量分解的人，从此张量在化学领域得到了充分的发展[23-25]。CP（CANDECOMP/PARAFAC）分解和 Tucker 分解是最常用的张量分解方法，这两种方法可以被认为是矩阵奇异值分解（SVD）和主成分分析（PCA）的高阶概括。CP 分解将张量分解为有限个秩一张量的总和，Tucker 分解沿着每个模态将张量分解为核心张量与因子矩阵的乘积。

在近几十年中，张量分解已经扩散到了各个领域。Lim 等[26]描述了在张量分解中使用压缩采样技术来对多个传感器的多个阵列的信号进行建模。通过设置一定程度的辐射源之间的间隔，可以始终保证代表信号的张量的最佳秩 r 近似的存在唯一性。用统一的方式处理具有无限连续字典、最低秩张量表示和盲源分离的最稀疏恢复问题。测量张量的分解以完全确定的方式同时定位和提取辐射源。Zhang 等[27]提出了一种鲁棒的 Tucker 张量分解（RTD）模型来处理被噪声信息遮挡的图像，并提出了一种简单但计算效率高的算法来解决基于 L1 范数的 Tucker 张量分解优化。通过 RTD 方法重建的图像比基于 PCA 的方法相比能够更有效地减少遮挡，需要的存储空间更少，分类精度更高。Cai 等[28]针对社交标签推荐提出了一个新的张量分解模型，专门用于处理非常稀疏的数据。该低秩张量分解模

中国知网 https://www.cnki.net

型利用低阶多项式独有的增强统计量的能力，来改善或增强用户，商品和标签之间的统计信息，同时避免过拟合。Wang 等[29]提出一种基于张量的低秩模型用于从单个降雨图像中去除降雨条纹。利用堆叠雨点来构造三阶张量，通过张量核范数保留降雨条纹的内在空间结构。

随着张量和张量分解的进一步发展，数据的尺寸和内容呈现出高维特性和高稀疏性，分解的存储和时间成本都大幅度增加。针对存储的问题，Papalexakis 等[30]提出一种利用随机采样技术加速张量分解的快速且可并行化的方法，产生稀疏的外积近似。确定每个模态的每个指标的重要性并作为度量，对高维张量进行有偏采样。采样张量代替原始张量进行 Parafac 分解，然后根据采样索引将因子向量重新分配到其原始位置，最终将所有不同的样本组合在一起输出因子矩阵。Sidiropoulos 等[31]将大张量随机压缩成较小的张量，如果大张量允许具有稀疏潜在因子的低秩分解，则随机采样可确保大张量与小张量的低秩分解的可识别性。如果潜在因子不稀疏，则可能无法保证低秩分解。

张量的高稀疏性使得计算过程中产生巨大的中间内存和时间成本，因此现有很多种计算方法来加速张量运算。MATLAB 支持各种按元素和二进制的密集张量元素。Bader 和 Kolda 引入了 MATLAB Tensor Toolbox[32]，它扩展了 MATLAB 以支持张量乘法和矩阵化。但是 MATLAB 无法支持以 CSC (Compressed Sparse Column) 格式存储的一组稀疏矩阵之外的张量形式。由于要选择以更细粒度进行计算的模态，Tensor Toolbox 的 Khatri-Rao 乘积作为中间数据将保存在工作内存中。Solomonik 等[33]提出了适用于分布式 CPU 系统的张量并行框架 CTF (Cyclops Tensor Framework)，它使用一维长整型数据对稀疏张量进行索引。因此对于超高维稀疏张量，CTF 可能会超出 64 位无符号长整型的范围。Kang 等[34]为 MapReduce 框架开发的并行 CPD-ALS 算法。通过将 MTTKRP 重新格式化为一系列 Hadamard 乘积来利用 MapReduce 的大规模并行性，在 Hadamard 乘积期间没有依赖关系，可以并行计算输出的每个元素。

SPLATT 是由 Smith 等[35]人编写的具有共享内存并行性的三维张量库，使用 CSF (Compressed Slice Fibers) 格式来利用张量的稀疏模式，具有较小的内存占用空间。Yang 等[36]分析了稀疏张量的多种存储格式，并开发了分段压缩策略来提高大型稀疏张量的存储效率。这种压缩策略可以避免在稀疏张量中存储大量的空切片和空纤维，从而大大减少了存储空间。每个张量被切成多个切面以形成一系列稀疏矩阵和向量乘法，从而形成流水线并行。Choi 等[37]通过分块优化技术来解决 MTTKRP 的瓶颈，使用维度分块和秩分块应用于 SPLATT 以改善性能，并将经过阻塞优化的 MTTKRP 内核与独特的处理器分区机制相结合，创建一个分布式 MTTKRP 实现。Hayashi 等[38]为密集张量的 MTTKRP 开发了共享内存并行算法，通过逐行算法来计算多个矩阵的 Khatri-Rao 乘积，将 MTTKRP 划分为两步并

中国知网 https://www.cnki.net

使用 OpenMP 和多线程 BLAS 的组合并行化。DFacTo 是由 Choi 等[39]为分布式张量分解而设计的算法，将 MTTKRP 表示为一系列的稀疏矩阵向量乘法，直接存储张量的切面而不是单个张量，可以使用任何标准稀疏线性代数库实现，处理大规模数据集。

1.2.2 国内研究现状

国内对张量分解的研究主要集中在应用上，针对算法本身的优化较少。廖志芳等[40]针对三部图中元素信息丢失以及标签系统中存在大量稀疏值和缺失值，提出了以三部图为基础的三维张量分解算法（TTD），分析三元关系之间的潜在关系进行用户标签预测。邹本友等[41]考虑到社会网络中用户之间的信任程度不同，设计了一种增量更新的张量分解算法用于用户信任推荐。该方法当有新用户加入时，只需要用原始张量结果和新增张量对结果进行动态更新。李贵等[42]通过历史标签构造原始张量，使用高阶奇异值分解方法对数据降维，挖掘多种类型实体之间的隐藏关联，提高算法的准确率和召回率。李喆康[43]考虑到交通流数据采样率不足导致的数据质量问题，提出了一种用于稀疏交通流数据补全的张量分解方法。在 CPWOPT 方法的基础上，加入体现路段的时变特征和限速特征的正则项，提高不同程度数据缺失下的补全效果。信号处理方面，张天骐等[44]提出一种多天线估计方法，结合张量分解与多项式库搜索。通过接收信号构造张量并分割为多个子张量，进行 CP 分解得到伪码片段和接收增益的因子矩阵，利用因子矩阵筛选得到每个用户的伪码序列。张德干等[45]提出一种基于张量的缺失估计方法，在贝叶斯张量分解的基础上结合随机采样，生成随机的数据子集再修复得到插补结果，将择优后的插补数据平均修复车辆交通缺失数据。郭伟等[46]提出一种 Frobenius 范数优化的高阶张量的奇异值分解方案，以张量的形式表示不同智能电网设备获取的大数据，设计一种张量的数据管理方法将数据降维，使用 Frobenius 范数来最小化重构误差。

1.3 本文研究内容

社会信息化的日益提高，各种信息系统采集了大量的数据。这种数据往往有着高维稀疏的特点，隐含着复杂行为的特殊性质。MTTKRP 作为基础的张量运算，其效率对张量分解有着显著影响。张量分解作为矩阵分解在高维空间上的推广，保留数据在高维空间的结构特征的同时将数据分解成若干性质简单的子张量。高维稀疏数据在做张量分解时会异常复杂与耗时，计算成本呈现指数级增长。本文以加速 CP 分解中的 MTTKRP 运算为目标，分析张量压缩格式对算法的影响，设计 CPU 下不同压缩格式的并行算法，结合 GPU 并行体系结构的特点与分层结构的压缩格式，通过调整线程执行方式提高并行程度，实现对数据的合并访存以及

中国知网 https://www.cnki.net

使用访存更快的共享内存等方式设计并实现了结合 GPU 的 MTTKRP 优化算法。

本文的主要工作和创新点如下:

1) 介绍高维稀疏张量的背景及研究意义，总结了张量的基本运算和 CP 分解的 ALS 算法，分析现有算法的计算瓶颈，提出优化分解过程中 MTTKRP 的优化目标。
2) 针对稀疏张量下 MTTKRP 算法的特点，设计 COO 格式与 CSF 格式下的并行代码，总结 MTTKRP 运算中可行的并行方式和并行粒度，基于纤维计算和 CSF 格式设计了基于 GPU 的 MTTKRP 算法。
3) 结合 GPU 的并行特点，给出切面并行、合并访存和使用共享内存等优化方式，并根据改进算法优化基于 CSF 压缩格式的 MTTKRP 算法。

1.4 论文结构安排

本文一共分为四章，简要说明如下：

第一章介绍了 MTTKRP 目前的研究背景、研究意义和研究现状。主要阐述了处理高维数据的 MTTKRP 的背景与遇到的问题，概括了张量和张量分解中对 MTTKRP 算法的研究，揭示了加速 MTTKRP 的意义，然后对国内外张量的应用场景以及张量分解中 MTTKRP 的加速方法的研究现状进行整理调查，在此基础上提出了本文的研究内容和创新点。

第二章介绍了本文采用的众核平台 GPU。对 CUDA 编程模型和存储模型进行详细了解，本文的算法优化工作主要基于 GPU/CUDA 实验平台。重点研究了张量和张量分解的基本概念。首先介绍了张量的基础运算以及相关矩阵乘法，然后详细介绍了 CP 分解的 ALS 算法中 MTTKRP 的运用场景。

第三章设计了 CPU 上的并行 MTTKRP 算法。首先介绍一些已有的稀疏张量存储格式，然后根据 COO 格式和 CSF 格式设计基于 OpenMP 的 MTTKRP 并行算法。分析不同格式下的算法的优缺点然后提出 GPU 并行的思路。

第四章对 MTTKRP 算法进行 GPU 上的优化设计。首先根据 CSF 格式的串行算法，给出 CSF 格式的 GPU 并行 MTTKRP 算法。利用 CUDA 编程模型，对 GPU 并行计算性能进行优化，采用的优化策略有：最大化线程执行并行度达到切面级并行；设计线程执行方式实现合并访存；优化存储器使用以增加片上内存利用率。最后通过现实数据集验证算法的合理性和有效性。

结论，总结了本文的工作，然后分析了本文尚未解决的一些问题以及未来的工作展望。

中国知网 https://www.cnki.net

第 2 章 相关理论基础

2.1 CUDA 模型

并行计算是在串行计算的基础上，将计算任务分解为若干个子任务，在同一时间使用不同的计算单元来执行若干指令。多组计算单元通过相互之间的通信与协作，以更快的速度共同完成一项大规模的计算任务[47]。GPGPU（for General-Purpose computing on the GPU）是基于 GPU 的通用计算方法。GPU 的计算能力对比 CPU 提高了数倍，因为其架构针对数据并行计算进行了优化。许多的用于 CPU 上的指令级并行的晶体管被替换为计算单元。GPU 的可编程性的提高使得传统的固定功能图形管道已经发展成为一个更加灵活和统一的多核体系结构，并引入了非图形数据并行编程语言。

2.1.1 CUDA 编程模型

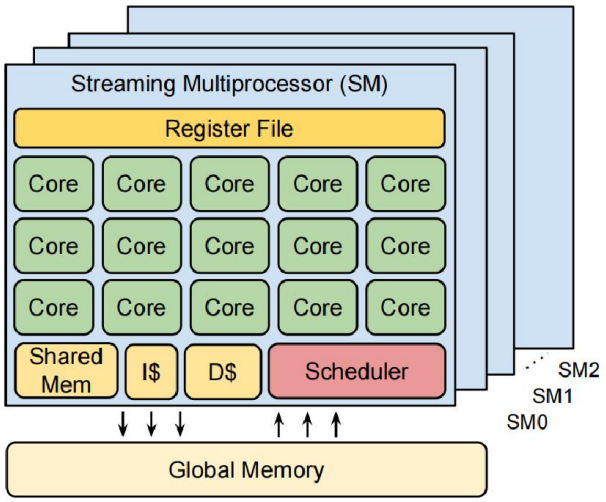


图 2.1 NVDIA GPU 架构

最初的 GPU 只应用于计算机图像显示。在 2003 年，研究人员开始使用 GPU 来加速并行程序运算，只能将程序改写去适用图形 API，Direct3D 或 OpenGL。当时两家主要的 GPU 供应商 NVIDIA 和 ATI 看到了他们设备的这种潜在替代用途，NVIDIA 的解决方案是发布 CUDA（Compute Unified Device Architecture）。图 2.1 表示 NVIDIA GPU 的主要架构，CUDA 是一种硬件和软件体系架构，它使 NVIDIA GPU 能够执行用 C，C++，Fortran 和其他语言编写的程序。CUDA 允许软件开发人员将 C 语言用作高级编程语言，使熟悉 C 编程语言的用户可以轻松编写设备可执行的程序。开发人员编写单一的源码，包含主机（CPU）和设备（GPU）代码。这两个部分由 CUDA 编译器自行分离和编译。它包括许多基本功

中国知网 https://www.cnki.net

能，允许直接控制 GPU 以及若干库函数，提供对大规模并行操作中常用的预编程操作的访问。

线程执行有不同的粒度。第一种执行粒度是由 32 个线程组成的线程束(warp)。线程束是 GPU 中的基本执行单元，warp 调度器依据线程束的状态进行调度，同一个线程束中所有的线程按照 SIMT 方式执行。第二种执行粒度被称为线程块（thread block），它最多可以包含 1024 个线程，其数量取决与所使用的硬件。最后一种粒度是由多个线程块组成的网格（grid)。一个网格包含执行同一个核函数的所有线程。这三种并行线程结构都有相应的三维内置变量来表示线程索引和维度大小。

CPU 拥有复杂的硬件以支持分支预测，它尝试猜测控制流指令（if, switch, do, for, while）的结果，并为最可能的结果做好准备。当执行控制流操作时，例如 if 语句，分支预测器会推断哪些条件最有可能满足，然后提前执行最有可能的结果所需要的操作。如果推断正确，操作已经完成，运行速度会显著提高。如果推断错误，CPU 会执行另一个分支，并产生延迟。GPU 没有复杂的分支预测机制，无法提前执行分支。GPU 以线程束为执行单位，同一线程束中的所有线程在同一周期必须执行相同的指令。当中有线程执行不同的指令，则产生不同的执行路径，这被称为线程束分化。分化发生时，GPU 禁用不满足条件的线程。当前分支执行结束，继续执行满足当前分支条件的线程，直到所有分支的线程聚合。这会增加线程束的指令总数，显著影响指令的吞吐量。为了获得最佳性能，应该使得同一线程束中的执行路径相同。

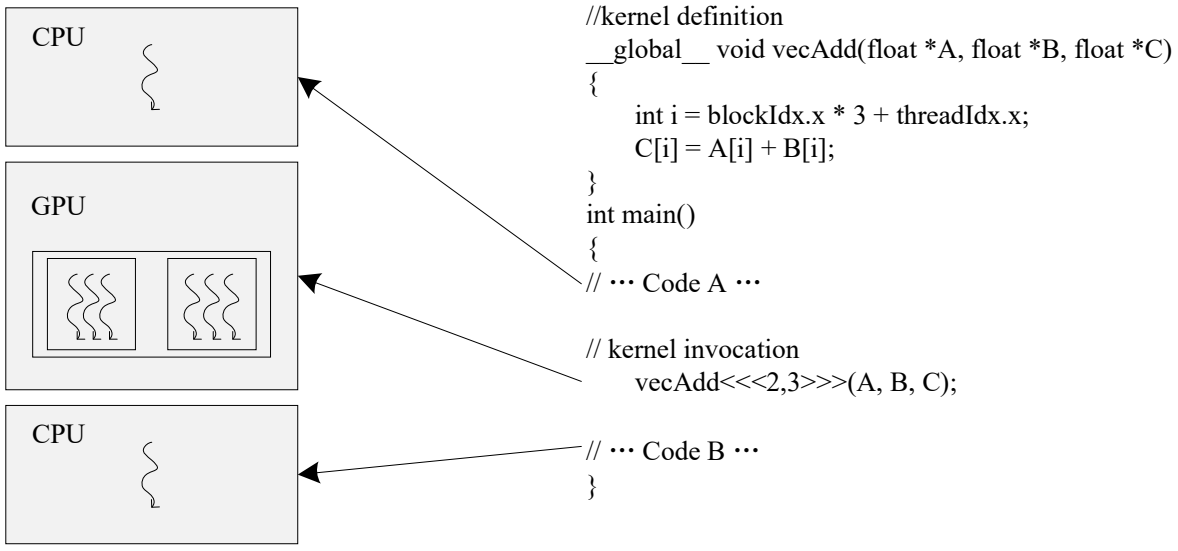


图 2.2 kernel 函数的示例

图 2.2 表示内核函数的调用过程。内核函数必须从主机上调用，并带有一组指明线程块大小和数量的配置参数。每个线程都执行内核函数的一个实例，并在其线程块中具有独特的线程号，可通过内置的 threadIdx 三维变量在核函数中确定

中国知网 https://www.cnki.net

当前线程号。线程块由并发执行的线程组成，这些线程可以通过块内屏障同步和共享内存在彼此之间进行协作。线程块在其网格内具有唯一的编号，由 blockIdx 确定。网格是由执行相同的核函数的线程块组成，图 2.2 中所有的线程执行 vecAdd 函数，这些线程组成当前程序的线程网格。

2.1.2 CUDA 存储模型

如图 2.3 所示[48]， GPU 的存储器主要有寄存器，本地内存，共享内存，常量内存，纹理内存和全局内存。每种类型的存储器都有优点和缺点以及不同的预期用途。GPU 可用的最底层级别的内存是寄存器（register）和本地内存（local memory），这些内存只能由内核函数中的单个线程访问。寄存器是 GPU 上访存延迟最低的内存，且只能由当前线程访问，用于存储自动变量。由于每个线程块上的寄存器数量有限，这意味着如果每个线程都需要大量的寄存器，那么可以启动的活跃线程就会减少。线程在内核中创建的数组很可能存储在本地内存中。若是线程在内核中创建本地数组，这些自动变量很有可能受制于寄存器数量有限被存储到本地变量中。每个线程都有用于寄存器溢出，函数调用等的私有本地内存（local memory）。本地内存访问与全局内存访问具有相同的高延迟和低带宽，因此很少使用本地内存。

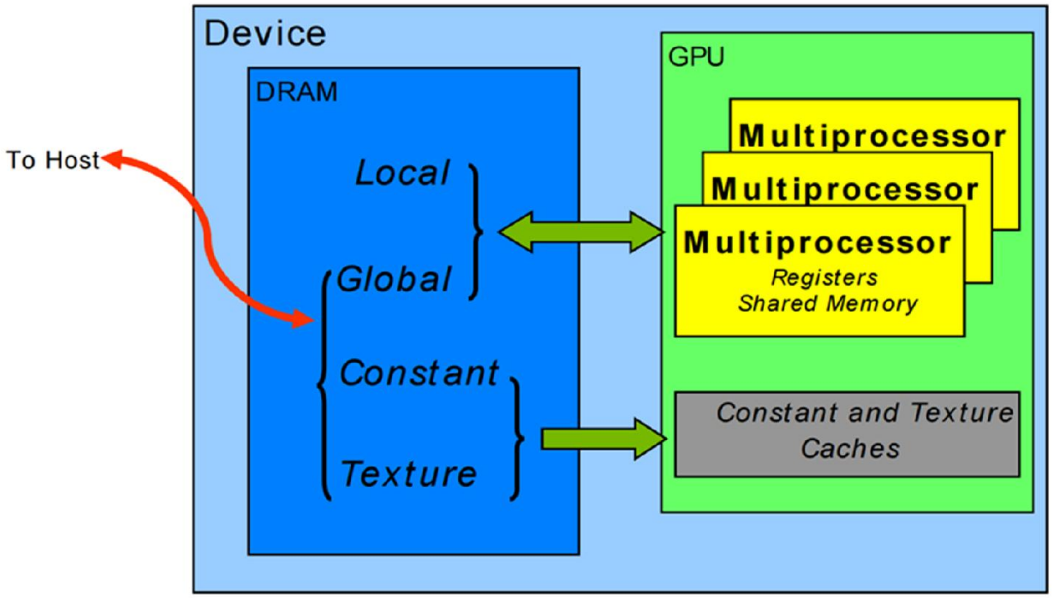


图 2.3 GPU 内存层次

每个线程块具有仅对该块的所有线程可见的共享内存（shared memory），且生存周期与线程块保持一致。共享内存属于片上内存，比本地内存和全局内存有更高的带宽和更低的延迟。共享内存使得同一个线程块上的线程能够协作通信，用于存储片上重用数据，可以大大减少对全局内存的访问。每个线程块指定所需共享内存的使用情况，当该线程块驻留在流多处理器（SM）上时，SM 会为其分

中国知网 https://www.cnki.net

配共享内存。因此如果线程块需要大量的共享内存，则同一 SM 可以同时调度的并发活跃线程块会减少，从而降低性能。共享内存由大小相等的可以被同时访问的n个内存模型构成，以实现并发访问的高内存带宽，他们被称为存储体（bank）。GPU 可以同时处理不同存储体上的数据加载和存储，产生的有效带宽是单个存储体带宽的n倍。一个线程束有 32 个线程，因此存储体的数量为 32。共享内存本质是一维地址空间，GPU 按行的分配方式将连续的地址映射到若干存储体中。线程束执行在共享内存上的数据加载或存储，且线程束中的每个线程访问的是不同的存储体，同时最多只访问一个地址，那么只需要一个内存事务即可完成，如图 2.4 所示。若是所有线程都访问同样的存储体上的同一地址，那么被访问的数据会被广播到所有请求的线程上。若是同一存储体上被多个线程访问不同地址，必须以串行的方式访问数据，访问时间更长且需要完成更多的内存事物，这被称为存储体冲突。只要每个线程访问不同的存储体，随机访问和顺序访问都不会造成存储体冲突。

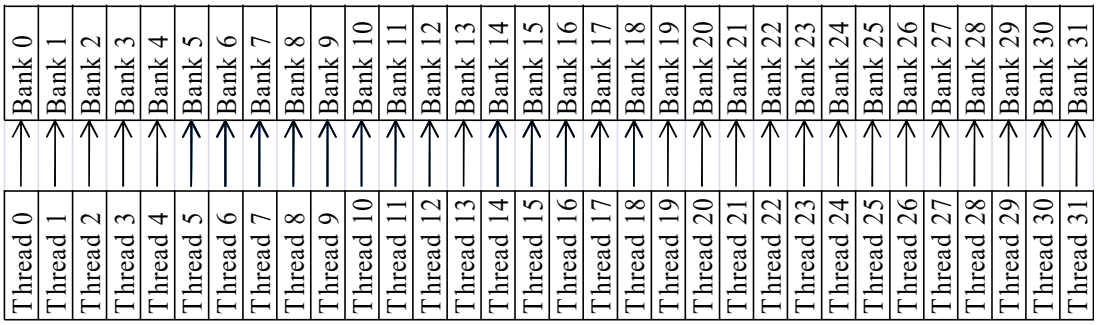


图 2.4 线程顺序访问存储体

常量内存（constant memory）和纹理内存（texture memory）是可以被所有线程访问的只读内存。常量内存是全局内存的一种只读的虚拟地址形式。纹理内存针对二维数据进行优化读取，有专门的只读 cache。常量和纹理内存都有特定的使用场景，其中每种情况都比全局内存快。当 warp 中的所有线程必须访问相同的数据时，常量内存是最佳选择。如果每个线程都需要在同一指令中访问值 x，则可以一次检索它并将其广播到所有线程。纹理内存可以合并内存访问以获取矩阵中彼此接近的值。例如一个 warp 中的 4 个线程访问分别访问 arr[x][y]、arr[x+1][y]、arr[x][y+1]和 arr[x+1][y+1]，纹理内存能够将它们组合到单个内存访问中。

全局内存（global memory）可以被网格中的所有线程访问。全局内存是访存最慢但容量最大的可读写内存，也是存储和检索数据的最简单内存。通过二级缓存来访问全局内存，一级缓存取决于 GPU 的架构和程序的设定。如果 GPU 只使用二级缓存，那么访存是以 32 字节的内存事务来实现的。如果也使用了一级缓存，内存事务变为 128 字节。当内存事务的第一个地址是事务大小的偶数倍时，就会产生对齐内存访问。合并访存是指线程束中所有的线程访问连续的对齐的内存块。根据访存是否对齐和合并，缓存加载的模式各有不同。在理想情况下，对

中国知网 https://www.cnki.net

齐与合并访存同时发生。线程束中所有线程请求的数据都在 128 字节的缓存行中且每个线程访问的数据不同，完成线程束所有的内存加载操作只需要一个内存事务，总线利用率是 100%。通过合并访存，在某种程度上可以优化全局内存访问，GPU 可以通过一条指令访问当前线程束要求的内存。

2.2 张量与张量运算

张量是向量和矩阵在高维空间上的抽象表达，使用张量表示数据可以保留数据的高维形式，并保留数据的相邻结构信息。阶数为N的张量是一个N维数组，它可以提供多路数据集的自然输入表示。

2.2.1 张量的基本概念

张量的表示是基于常用的矩阵表示，它可以扩展到一般张量。张量存在维数(dimension)的概念，也可以被称为模态(mode)或者是阶(order)[49]。矩阵有两种模态，矩阵的行与列，因此访问矩阵的元素需要两个索引变量i和j。矩阵的行可以通过$A(i,:)$进行访问，其中行索引变量i保持固定，列索引变量j在整个列维度范围内。同样，$A(:,j)$通过j在保持恒定的同时改变i来访问列。

张量可以由任意数量的维度组成，拥有任意数量的索引变量。本文的主要工作集中在三阶张量，只需要三个索引来访问张量数据，对张量$\mathcal{X}$通过索引值i，j，k的元素来访问$\mathcal{X}(i,j,k)$。图 2.5 表示一个三维张量。

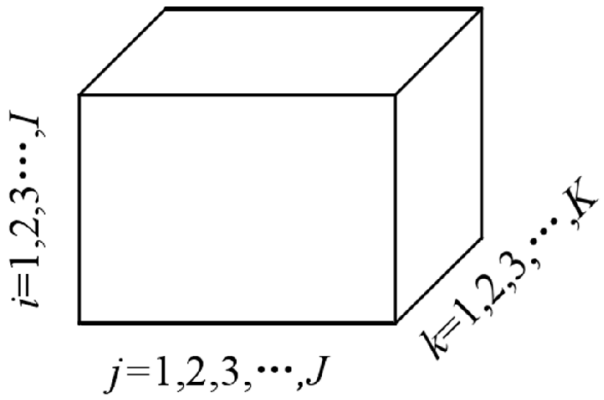
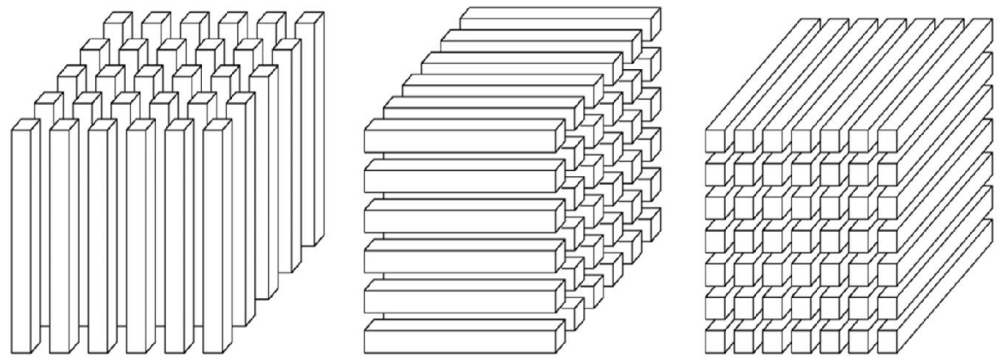


图 2.5 三维张量的表示

与矩阵一样，可以通过固定一些索引并改变其它索引来选择张量的范围。固定一个索引变量，遍历其余两个索引可以得到三维张量的矩阵切面(slice)。若是固定两个可以得到向量，也被称之为纤维(fiber)。张量的部分运算需要指定运算模态。将张量视为一系列切面和纤维的组合有利于将张量运算概念化，可以沿着任意坐标轴产生切面和纤维。图 2.6 中三维张量只遍历一种索引可以产生三

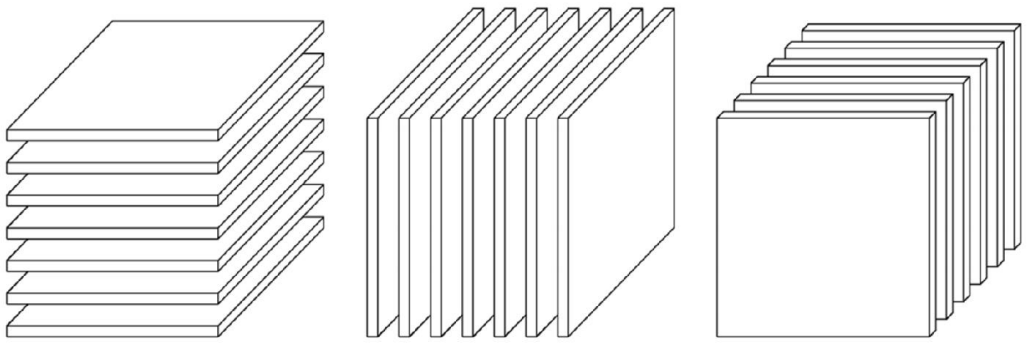
中国知网 https://www.cnki.net

种模态的纤维，分别称为行$X_{:jk}$（column）、列$X_{i:k}$（row）和管$X_{ij:}$（tube）。同理图 2.7 中只固定一种索引可以产生三种切面，分别为水平切面$X_{i::}$（horizontal slice）、横向切面$X_{:j:}$（lateral slice）和正向切面$X_{::k}$（frontal slice），可以将正向切面更紧凑地表示为X_k。



(a) Mode-1 (column) fibers: $\mathbf{x}_{:jk}$ (b) Mode-2 (row) fibers: $\mathbf{x}_{i:k}$ (c) Mode-3 (tube) fibers: $\mathbf{x}_{ij:}$

图 2.6 三维张量的纤维



(a) Horizontal slices: $\mathbf{X}_{i::}$ (b) Lateral slices: $\mathbf{X}_{:j:}$ (c) Frontal slices: $\mathbf{X}_{::k}$ (or $\mathbf{X}_k$)

图 2.7 三维张量的切面

2.2.2 张量的基础运算

张量矩阵化通过将张量展平为二维的矩阵。根据张量划分的概念来理解，通过合并张量的切面生成矩阵。矩阵化取决于模态，一个有 n 种模态的张量可以得到 n 种展开后的矩阵。

定义张量$\mathcal{X} \in \mathrm{R}^{I_1 \times I_2 \times \cdots \times I_n}$的模 n 展开为$X_{(n)}$，此时模 n 的纤维被排列组合成展开矩阵的列。其中张量$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_n}$的$(i_1, i_2, \ldots, i_n)$对应位置的元素在模 n 矩阵中的位置为$(i_n, j)$，满足

$$j = 1 + \sum_{k=1, k \neq n} (i_k - 1) J_k \tag{2.1}$$

中国知网 https://www.cnki.net

$$J_k = \prod_{m=1,m\neq n}^{k-1} I_m \tag{2.2}$$

例如现有一个三维张量$\mathcal{X} \in \mathbb{R}^{3\times4\times2}$，它的正向切面为

$$X_1 = \begin{bmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{bmatrix}, X_2 = \begin{bmatrix} 13 & 16 & 19 & 22 \\ 14 & 17 & 20 & 23 \\ 15 & 18 & 21 & 24 \end{bmatrix} \tag{2.3}$$

模 n 展开为

$$X_{(1)} = \begin{bmatrix} 1 & 4 & 7 & 10 & 13 & 16 & 19 & 22 \\ 2 & 5 & 8 & 11 & 14 & 17 & 20 & 23 \\ 3 & 6 & 9 & 12 & 15 & 18 & 21 & 24 \end{bmatrix}$$

$$X_{(2)} = \begin{bmatrix} 1 & 2 & 3 & 13 & 14 & 15 \\ 4 & 5 & 6 & 16 & 17 & 18 \\ 7 & 8 & 9 & 19 & 20 & 21 \\ 10 & 11 & 12 & 22 & 23 & 24 \end{bmatrix}$$

$$X_{(3)} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 \end{bmatrix}$$

因张量与矩阵的维数不同，张量矩阵乘法（Tensor-Times-Matrix, TTM）将输入张量切成一组矩阵，并且对乘数矩阵执行一系列矩阵乘法，最后得到的结果仍为张量。如图 2.7 所示，张量的切片方式取决于运算要求的模态并且决定结果张量的形式。

定义张量$\mathcal{X} \in \mathbb{R}^{I_1\times I_2\times\cdots\times I_N}$与矩阵$\mathrm{U} \in \mathbb{R}^{J\times I_n}$，模 n（mode-n）乘法记作$\mathcal{X} \times_n \mathrm{U}$且大小为$I_1 \times \cdots \times I_{n-1} \times J \times I_{n+1} \times \cdots \times I_N$，其中

$$(\mathcal{X} \times_n U)_{i_n\cdots i_{n-1} j i_{n+1}\cdots i_N} = \sum_{i_n=1}^{I_n} x_{i_1 i_2\cdots i_N} u_{j i_n} \tag{2.4}$$

不难看出，模 n 乘积其实就是模 n 纤维与矩阵U的乘积。公式(2.4)可以改写成张量的矩阵化形式

$$\mathcal{Y} = \mathcal{X} \times_n U \Leftrightarrow Y_{(n)} = UX_{(n)} \tag{2.5}$$

例如式(2.3)所示的张量$\mathcal{X} \in \mathbb{R}^{3\times4\times2}$，设有乘数矩阵$\mathrm{U} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$，则 mode-1 下张量矩阵乘法$\mathcal{Y} = \mathcal{X} \times_1 U \in \mathbb{R}^{3\times4\times2}$为

$$Y_1 = \begin{bmatrix} 14 & 32 & 50 & 68 \\ 32 & 77 & 122 & 167 \\ 50 & 122 & 194 & 266 \end{bmatrix}, Y_2 = \begin{bmatrix} 86 & 104 & 122 & 140 \\ 202 & 257 & 302 & 347 \\ 338 & 410 & 482 & 554 \end{bmatrix}$$

mode-n 张量矩阵乘法满足一些条件下的乘法性质，多种模态下的矩阵乘积对乘法顺序式无关。即

$$\mathcal{X} \times_m A \times_n B = \mathcal{X} \times_n B \times_m A \ (m \neq n) \tag{2.6}$$

中国知网 https://www.cnki.net

如果矩阵的模态是一样的，式(2.6)可以写成

$$\mathcal{X} \times_n A \times_n B = \mathcal{X} \times_n (BA) \tag{2.7}$$

在张量运算中，Kronecker 乘积，Khatri-Rao 乘积与 Hadamard 乘积是常要用到的矩阵运算。

设有矩阵$A \in \mathbb{R}^{I \times J}$和矩阵$B \in \mathbb{R}^{K \times L}$，定义 Kronecker 乘积$A \otimes B$为

$$\begin{aligned} \mathrm{A} \otimes B &= \begin{bmatrix} a_{11}B & a_{12}B & \cdots & a_{1J}B \\ a_{21}B & a_{22}B & \cdots & a_{2J}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{I1}B & a_{I2}B & \cdots & a_{IJ}B \end{bmatrix} \\ &= [a_1 \otimes b_1 \quad a_1 \otimes b_2 \quad a_1 \otimes b_3 \quad \cdots \quad a_J \otimes b_{L-1} \quad a_J \otimes b_L] \end{aligned} \tag{2.8}$$

结果矩阵大小为$(IK) \times (JL)$。

设有矩阵$A \in \mathbb{R}^{I \times K}$和矩阵$B \in \mathbb{R}^{J \times K}$，定义 Khatri-Rao 乘积$A \odot B$为

$$A \odot B = [a_1 \otimes b_1 \quad a_2 \otimes b_2 \quad \cdots \quad a_K \otimes b_K]$$

其中a_k和b_k，$r = 1, \cdots, K$是矩阵A和B的列向量，结果矩阵大小为$(IJ) \times K$。

Khatri-Rao 乘积可以看作是具有相同列数矩阵的按列匹配的 Kronecker 乘积的组合。如果用向量代替矩阵，那么 Khatri-Rao 乘积和 Kronecker 乘积是一致的，即$a \odot b = a \otimes b$。

设有相同大小矩阵A和B，其维数为$I \times J$，定义 Hadamard 乘积$A * B$为

$$\mathrm{A} * B = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \cdots & a_{1J}b_{1J} \\ a_{21}b_{21} & a_{22}b_{22} & \cdots & a_{2J}b_{2J} \\ \vdots & \vdots & \ddots & \vdots \\ a_{I1}b_{I1} & a_{I2}b_{I2} & \cdots & a_{IJ}b_{IJ} \end{bmatrix}$$

上述矩阵操作还具有以下数学性质：

$$\begin{gathered} (A \otimes B)(C \otimes D) = AC \otimes BD \\ (A \otimes B)^{\dagger} = A^{\dagger} \otimes B^{\dagger} \\ A \odot B \odot C = (A \odot B) \odot C = A \odot (B \odot C) \\ (A \odot B)^T (A \odot B) = A^T A * B^T B \\ (A \odot B)^{\dagger} = \left((A^T A) * (B^T B)\right)^{\dagger} (A \odot B)^T \end{gathered} \tag{2.9}$$

†表示矩阵的摩尔－彭若斯广义逆。

2.3 CP 分解算法

张量能够完整直接表示高维数据，并且保持数据在高维空间的结构特征。张量分解的目的是将原始张量分解为一系列张量，以提炼出张量潜在的模式和因子。经典张量分解方法有 CP 分解，Tucker 分解和 HOSVD 分解等等，本文主要关注 CP 分解。

设有张量$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$，若$\mathcal{X}$可以写成$N$个向量的外积即

$$\mathcal{X} = a^{(1)} \circ a^{(2)} \circ \cdots \circ a^{(N)} \tag{2.10}$$

中国知网 https://www.cnki.net

称$\mathcal{X}$为秩一张量，其中$a^{(i)} \in R^{I_i}, i = 1, \cdots, N$，”∘”表示向量外积。此时张量的每个元素可以表示为相应矩阵的元素的乘积，即

$$x_{i_1 i_2 \cdots i_N} = a_{i_1}^{(1)} a_{i_2}^{(2)} \cdots a_{i_N}^{(N)}$$

图 2.8 表示一个三维的秩一张量$\mathcal{X} = a \circ b \circ c$，其中$x_{ijk} = a_i b_j c_k$。

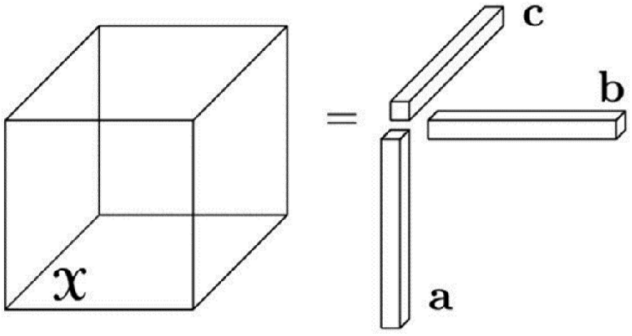


图 2.8 三维秩一张量

定义张量的秩R为满足秩一张量之和等于该张量的最少秩一张量数，即

$$\mathcal{X} = \sum_{r=1}^{R} a_r^{(1)} \circ a_r^{(2)} \circ \cdots \circ a_r^{(N)} \tag{2.11}$$

张量与矩阵对于秩的定义是类似的，但是张量在实数域和复数域上的秩可能不同。由于数据的高维特性，目前未有直接的算法可以求出给定张量的秩，实际上求张量的秩是一个 NP-hard 问题。张量分解的主要目的是把原始张量分解为若干特征更加明显的张量，因此分解的秩通常比原始张量的维数小得多。这确保得到的因子矩阵更易于解释，并且较小的因子矩阵减少矩阵的构造操作。

CP 分解是奇异值分解（SVD）在高维数据空间的上的扩展。SVD 将一个较复杂的矩阵分解成若干较小较简单的子矩阵乘积。设有矩阵$A \in \mathbb{R}^{m \times n}$,定义$A$的奇异值分解为：

$$A = U \Sigma V^T$$

其中$\Sigma \in \mathbb{R}^{m \times n}$是一个对角矩阵，$U \in \mathbb{R}^{m \times m}$和$V \in \mathbb{R}^{n \times n}$是酉矩阵，即

$$U^T U = I, V^T V = I。$$

CP 分解将这一概念扩展为将张量分解为R个秩一张量的总和，将张量$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$近似为

$$\mathcal{X} \approx \sum_{r=1}^{R} u_r^{(1)} \circ \cdots \circ u_r^{(N)} \tag{2.12}$$

其中R表示张量的秩，表示作分量的秩一张量的个数，因此 CP 分解受到张量的秩的影响，称为秩R的 CP 分解。在低秩近似中，R的值通常是小于 100。向量$u_r^{(1)} \circ \cdots \circ u_r^{(N)}$的外积产生$R$个秩一张量。称$U^{(n)} \in \mathbb{R}^{I_n \times R}, n = 1, \cdots, N$被称为因子矩阵，由秩一张量分解得到的向量组成，即

中国知网 https://www.cnki.net

$$U^{(n)} = \left[u_1^{(n)} u_2^{(n)} \cdots u_R^{(n)}\right]$$

若对这些向量进行归一化处理，那么公式(2.12)可以写成

$$\mathcal{X} \approx \sum_{r=1}^{R} \lambda_r u_r^{(1)} \circ \cdots \circ u_r^{(N)} \equiv \left[\lambda; U^{(1)}, \cdots, U^{(N)}\right]$$

得到所有矩阵的因子权重为

$$\lambda = \{\lambda_1, \cdots, \lambda_r\}$$

如图 2.9 所示以三维张量为例，给定以一个大小为$I \times J \times K$的张量$\mathcal{X}$，其关于秩R的 CP 分解为

$$\mathcal{X} \approx \hat{\mathcal{X}} = \sum_{r=1}^{R} A(:,r) \circ B(:,r) \circ C(:,r) \tag{2.13}$$

其中矩阵$A \in \mathbb{R}^{I \times R}$，$B \in \mathbb{R}^{J \times R}$，$C \in \mathbb{R}^{K \times R}$为张量$\mathcal{X}$的因子矩阵。

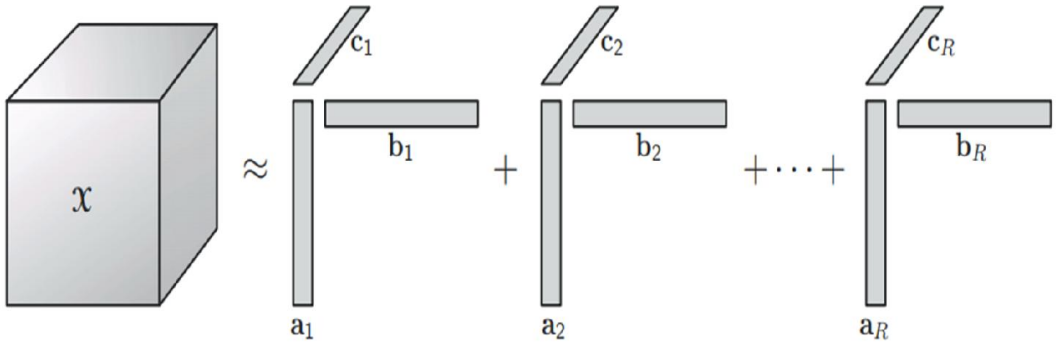


图 2.9 三维张量的 CP 分解

张量位于(i,j,k)上的元素可以分解为

$$x_{ijk} \approx \hat{x}_{ijk} = \sum_{r=1}^{R} a_{ir}\, b_{jr} c_{kr} = \sum_{r=1}^{R} A(i,r) \circ B(j,r) \circ C(k,r)$$

其中$i = 1, \cdots, I$，$j = 1, \cdots, J$，$k = 1, \cdots, K$。

将张量$\mathcal{X}$按照不同 mode 展开，可以将公式(2.13)写成

$$X_{(1)} \approx A(C \odot B)^T$$
$$X_{(2)} \approx B(C \odot A)^T$$
$$X_{(3)} \approx C(B \odot A)^T$$

计算 CP 分解的最流行的算法是交替最小二乘法（Alternating Least Squares Method, ALS），以下简称 CP-ALS。CP-ALS 的每次迭代都会遍历所有的模态，并更新每个模态的因子矩阵，同时保持其它因子矩阵不变。重复执行此过程，直到满足指定的收敛精度或者是最大迭代次数为止。

现有一个三维张量$\mathcal{X} \in \mathbb{R}^{I \times J \times K}$，其秩$R$的 CP 分解可以转化为寻找一个近似张量$\hat{\mathcal{X}}$，满足

中国知网 https://www.cnki.net

$$min_{\hat{\mathcal{X}}}\|\mathcal{X}-\hat{\mathcal{X}}\|$$
$$s.t.\hat{\mathcal{X}}=\sum_{r=1}^{R}A(:,r)\circ B(:,r)\circ C(:,r)$$

按照张量的模态展开可以得到

$$min_A\left\|X_{(1)}-A(C\odot B)^T\right\|_F$$
$$min_B\left\|X_{(2)}-B(C\odot A)^T\right\|_F$$
$$min_C\left\|X_{(3)}-C(B\odot A)^T\right\|_F$$

如算法 2.1 所示，CP-ALS 先固定B和C去更新A，再固定A和C去更新B，最后固定A和B去更新C。以此作为一次迭代，不断重复直到满足收敛条件。

算法 2.1 关于秩 R 的三维张量的 CP-ALS 算法

输入：张量$\mathcal{X}\in\mathbb{R}^{I\times J\times K}$，秩$R$

输出：矩阵A，B，C

1. 初始化矩阵$A\in\mathbb{R}^{I\times R}$，$B\in\mathbb{R}^{J\times R}$，$C\in\mathbb{R}^{K\times R}$
2. do
3. $A=X_{(1)}(C\odot B)(C^TC*B^TB)^{\dagger}$;
4. $B=X_{(2)}(C\odot A)(C^TC*A^TA)^{\dagger}$;
5. $C=X_{(3)}(B\odot A)(B^TB*A^TA)^{\dagger}$;
6. if $\left\|X_{(1)}-A(C\odot B)^T\right\|_F$满足精度要求
   break;
7. while true;
8. return A,B,C

以 mode-1 为例，此时矩阵A可以写成

$$A=X_{(1)}[(C\odot B)^T]^{\dagger} \tag{2.14}$$

根据公式(2.9)可以将上式改写成

$$A=X_{(1)}[(C\odot B)^T]^{\dagger}=X_{(1)}(C\odot B)(C^TC*B^TB)^{\dagger}$$

由于秩R通常选取较小值，这样的改写将原本大小为$JK\times R$的矩阵求伪逆的运算转化成大小为$R\times R$的矩阵求伪逆的运算，有效降低计算量。$(C^TC*B^TB)^{\dagger}$是一个足够小的$R\times R$的矩阵，他的计算成本相对较小。因此大部分计算工作来自矩阵化张量乘以矩阵 Khatri-Rao 乘积（Matricized-Tensor times Khatri-Rao product, MTTKRP），$X_{(1)}(C\odot B)$，如何优化 MTTKRP 是本文的主要工作。

2.4 本章小结

本章首先对 CUDA 的编程模型和内存模型作了简要介绍，然后详细介绍了张

中国知网 https://www.cnki.net

量与张量分解的基本概念。着重介绍了 CP 分解以其经典的 ALS 算法，通过不断遍历所有的模态，并更新每个模态的因子矩阵，同时保持其它因子矩阵不变。分析得到 MTTKRP 计算是 CP 分解的 ALS 算法中的计算瓶颈，为后文实现 MTTKRP 并行化提供基础。

中国知网 https://www.cnki.net

第 3 章 基于 CPU 的 MTTKRP 算法分析

3.1 张量的压缩格式

对于大部分的元素都为零的张量，稀疏性意味着可以使用紧凑的存储格式来减少这些零元素的存储。张量分解往往涉及到高维稀疏张量，现有的密集张量计算无法适应稀疏数据集。高稀疏性使得重复的数据消耗大量的计算资源且高维度使得内存占用成倍增加。为了满足高维张量的稀疏性，以含有 8 个非零元的张量$\mathcal{A} \in R^{3\times3\times3}$为例，其 mode-1 展开为

$$A_{(1)} = \begin{bmatrix} 4 & 0 & 0 & 6 & 0 & 0 & 0 & 0 & 1 \\ 0 & 5 & 0 & 0 & 0 & 0 & 2 & 0 & 0 \\ 7 & 0 & 0 & 0 & 3 & 0 & 0 & 0 & 4 \end{bmatrix}$$

介绍几种常用的稀疏张量存储格式。

3.1.1 COO 格式

COO（Coordinate）格式是一种简单又灵活的存储方案，具有用于稀疏矩阵的三元组（行，列，值）。行，列和值的数组分别存储非零元的行索引，列索引和值。COO 格式可以扩展到高维数据，该数据使用四元组存储三阶张量。数组x，y，z，$vals$分别存储张量非零元的第一维，第二维，第三维的索引以及值。对含有nnz个非零元的张量$\mathcal{X}$且大小为$I \times J \times K$，所需要存储空间大小为$4nnz$。例如$\mathcal{A}$的 COO 格式为

$$x = (0,0,2,0,2,0,1,2)$$
$$y = (0,1,0,0,1,2,0,2)$$
$$z = (0,0,0,1,1,2,2,2)$$
$$vals = (4,5,7,6,3,1,2,4)$$

COO 格式不会压缩张量，但会通过删除所有零元来减少存储量。COO 允许将新的非零元直接插入未排序的张量中，各非零元之间相对独立。

3.1.2 CSR 格式

CSR 格式是用于矩阵压缩的常用格式，列索引和非零元存储在数组$cols$和$vals$中，每行第一个非零元在数组$cols$的起始位置存储到row_ptr中。对于一个$N \times M$矩阵，row_ptr的长度为$N+1$，并将第i行的偏移量存储在$row_ptr[i]$中。对含有nnz个非零元的张量$\mathcal{X}$且大小为$I \times J \times K$，CSR 格式需要的存储空间大小为$I \times J + 2nnz$。

稀疏矩阵的 CSR 格式可以扩展到三维稀疏张量上，将张量$\mathcal{X}$按照 mode-1 展开得到$X_{(1)}$。定义两个数组z和$vals$分别存储第三维索引和非零元的值以及数组row_ptr代表数组z中矩阵$(\mathcal{X}_{1::}^T \mathcal{X}_{2::}^T \cdots \mathcal{X}_{I::}^T)^T$每一行非零元的起始位置。$\mathcal{A}$的 mode-1

中国知网 https://www.cnki.net

下 CSR 格式为

$$row_ptr = (0,1,2,3,4,5,6,7,8)$$
$$z = (0,0,2,1,0,0,1,2)$$
$$vals = (4,6,1,5,2,7,3,4)$$

3.1.3 CSF 格式

压缩张量的主要目标就是减少多个非零元共享的索引值重复。Smith[50]等人扩展稀疏矩阵的 CSR 格式，提出了最通用的稀疏张量压缩格式 CSF (Compressed Sparse Fibers)。CSF 通过分层索引指针数组来扩展 CSR，以纤维为中心来压缩其它张量维度，从而减少索引值的重复。这种分层技术可以被视作构建张量的树状表达。

对于三维张量$\mathcal{X}$，从每个$\mathcal{X}_{i::}$切面可以形成一个深度为 4 的树。第m层的结点的值为该张量在第m个维度的索引值，从根结点到一个叶子结点的路径构成一个非零元在该张量中的位置索引，并且非零元的值也存储在叶子结点中。例如张量$\mathcal{A}$的 CSF 格式为

$$slc_idx = (0,1,2)$$
$$slc_ptr = (0,3,5,8)$$
$$fbr_idx = (0,1,2,0,1,0,1,2)$$
$$fbr_ptr = (0,1,2,3,4,5,6,7,8)$$
$$innerfbr_idx = (0,0,2,1,0,0,1,2)$$
$$vals = (4,6,1,5,2,7,3,4)$$

张量的 CSF 格式是树形结构，一个叶子结点表示一个非零元，而兄弟叶子结点形成了纤维，令除了最后一个索引之外所有索引保持不变。对同一张量使用两种不同的压缩顺序对 CSF 也有一定的影响。矩阵可以选择压缩行或者是压缩列，高维张量也具有更多的压缩顺序。对于具有s个切面，f个纤维和nnz个非零元的三维张量，需要$2s + 2f + nnz$的索引值空间。

3.2 COO 格式下 MTTKRP 并行算法分析

CP-ALS 算法是 CP 分解最常用的方法之一， 其中 MTTKRP 根据一个输入张量和两个输入矩阵产生一个密集矩阵。作为 CP 分解中的计算瓶颈，如何提高 MTTKRP 并行效率为提升分解速度的关键。COO 格式作为最简单的数据格式，存储了每个非零元的值及所有位置索引。COO 格式不偏向任何模态，是一种中性的存储方式。

假设有一秩为R三维张量$\mathcal{X} \in \mathbb{R}^{I\times J\times K}$，有$nnz$个非零元，以 mode-1 为例，定义

$$M = X_{(1)}(C \odot B) \tag{3.1}$$

中国知网 https://www.cnki.net

为矩阵化张量乘 Khatri-Rao 积（MTTKRP），MTTKRP 在 CP-ALS 中每次迭代的每个模态都会执行一次，更新当前模态下的因子矩阵。定义矩阵B和C的行数为J和K，M的行数为I，这三个矩阵的列数都为R。尽管得到的结果矩阵M大小为$I \times R$，但是$C \odot B$是一个稠密的$JK \times R$的矩阵，且需要的中间内存比张量本身要大得多。因此在大多数情况下不会直接计算$C \odot B$的结果，常通过将 Khatri-Rao 乘积展开，矩阵元素与非零元直接进行计算，减少中间过程。

公式(3.1)按元素表示为

$$M(i,r) = \sum_{z=0}^{JK} X_{(1)}(i,z)B(z\%J,r)C(z/J,r)$$

将$X_{(1)}$按照张量元素展开得到

$$M(i,r) = \sum_{k=0}^{K}\sum_{j=0}^{J} \mathcal{X}(i,j,k)\big(B(j,r) * C(k,r)\big) \tag{3.2}$$

这一步骤将$X_{(1)}$的分成J和K两部分，计算M的一个元素需要进行$2 * J * K$次计算。此时 MTTKRP 的计算时间[51]约为

$$T_{MTTKRP} = O(N^{\epsilon} nnz R)$$

其中ϵ取决于稀疏张量的特征，如模态大小和非零元分布。Tensor Toolbox 的ϵ为 1，而 SPLATT 和 DFacTo 在三维张量上的$\epsilon \approx 0$，在更高维张量上$\epsilon \in (0,1)$。MTTKRP 在 CP-ALS 算法中占有 69%-99%的计算时间。

算法 3.1 基于 COO 格式的 MTTKRP 算法

输入：张量$\mathcal{X} \in \mathbb{R}^{I \times J \times K}$，秩$R$，稠密矩阵$B \in \mathbb{R}^{J \times R}$，$C \in \mathbb{R}^{K \times R}$，非零元个数$nnz$

输出：更新的稠密矩阵 M

```
1. for n=1→nnz do
2.     i=x[n], j=y[n], k=z[n]
3.     for r=1→R do
4.         M[i, r] + = vals[n] * C[k][r] * B[j][r]
5.     end
6. end
7. return A
```

算法 3.1 是基于 COO 格式的 MTTKRP 算法实现，MTTKRP 将位置(i,j,k)上的非零元与B的第j行和矩阵C的第k行相乘，然后将结果累加到矩阵A的第i行上。第 1 行将n为当前索引来遍历张量所有的非零元，第 2 行获取非零元在张量中的索引值，第 3-4 行锁定当前非零元，非零元与其索引以及输入矩阵执行实际计算。

中国知网 https://www.cnki.net

观察 COO 的算法流程发现，可以考虑基于矩阵列或非零元的并行方式。若是直接对非零元进行并行化，对于部分索引相同的非零元会产生读写冲突，需要设置同步互斥操作。将结果矩阵的求和设置为临界区，对结果矩阵的写操作同一时刻只能有一个线程操作。此时，求和操作是串行执行的，严重降低并行效率。因此利用 OpenMP[52]直接按列对算法 3..1 进行并行化，如算法 3.2 所示。按列并行的方式避免了对结果矩阵的读写冲突。此时每个线程需要访问张量全部的非零元，线程的负载可能过重。

算法 3.2 基于 COO 格式的并行 MTTKRP 算法

输入：张量$\mathcal{X} \in \mathbb{R}^{I \times J \times K}$，秩$R$，稠密矩阵$B \in \mathbb{R}^{J \times R}$，$C \in \mathbb{R}^{K \times R}$，非零元个数$nnz$

输出：更新的稠密矩阵 A

```
1. #pragma omp parallel for
2. for r=1→R do
3.     for n=1→nnz do
4.         i=x[n], j=y[n], k=z[n]
5.         A[i, r] + = vals[n] * C[k][r] * B[j][r]
6.     end
7. end
8. return A
```

观察 COO 格式下的并行算法发现有如下的特征：

1. 计算选用的矩阵B和C的元素与非零元的索引相关，当非零元之间有相同的索引j或k时，对应的矩阵元素是相同的。
2. COO 格式下的索引只参与一次计算，无法利用处于相同纤维或切面的非零元的潜在关联。
3. 矩阵C在计算求和过程中重复读取，C与内部求和无关。

公式(3.2)得出矩阵C与内部求和无关，将矩阵C的计算从内部的求和提取到外部求和内可得

$$M(i,r) = \sum_{k=0}^{K} C(k,r) * \sum_{j=0}^{J} \mathcal{X}(i,j,k) B(j,r) \tag{3.3}$$

此时计算一个元素所需要的操作减少为$(J+1)*K$，节省了$(J-1)*K$次操作。

图 3.1 说明了在 mode-1 下 MTTKRP 在张量上的运算抽象表示。把张量的纤维和矩阵的列视为向量，每当计算结果矩阵的一个元素，MTTKRP 进行了两次向量内积的操作。遍历张量切面中所有的纤维，每个纤维与矩阵B中指定的列进行向量内积操作得到的结果

中国知网 https://www.cnki.net

$$temp(k) = \sum_{j=0}^{J} \mathcal{X}(i,j,k)B(j,r) \tag{3.4}$$

可以组成一个中间向量$temp$。这个中间向量再与矩阵C中r列进行向量内积操作得到结果矩阵M位于(i,r)上的元素。

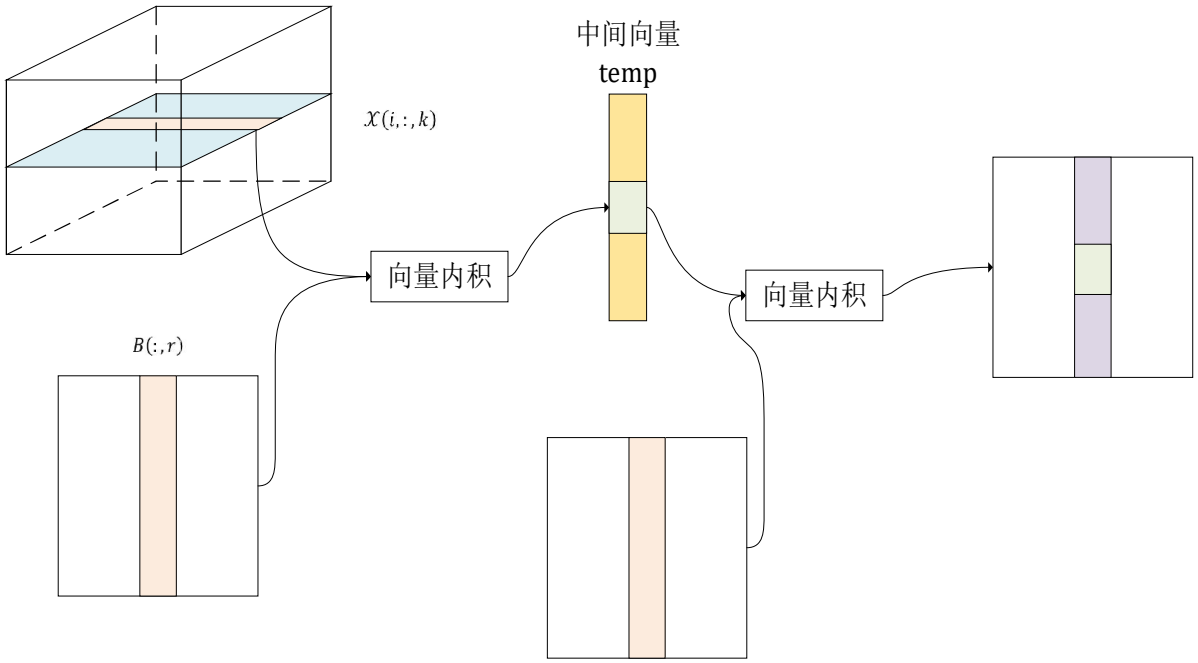


图 3.1 MTTKRP 的运算过程

因此可以考虑使用提高索引利用率的 CSF 格式存储张量，将计算对象从以非零元为单位转换成以纤维或切面为单位，结合公式(3.4)设计并行度更高的 MTTKRP 优化算法。

3.3 CSF 格式下 MTTKRP 并行算法分析

COO 格式作为一种数据独立性强且并行性较高的存储格式，无法有效利用到处于同一切面或同一纤维上的非零元数据相关性，这些重复的索引可以有效的减少数据的重复读取，提高算法的计算效率。图 3.1 表明可以通过对张量按切面对纤维进行压缩，同一个切面上的非零元可以共用一个索引值，同一个纤维上的非零元共享当前的纤维对应的索引值。CSF 格式作为一种分层的以纤维为中心的压缩格式，有效提高了索引利用率。CSF 格式把张量分成切面和纤维两个层次，记录非空的切面和纤维及非零元的索引值。

算法 3.3 表示 CSF 格式下 mode-1 的 MTTKRP 算法，共四个循环，分别遍历结果矩阵的所有列，张量的所有非空切面，每个非空切面上所有的非空纤维，以及每个非空纤维上的非零元。第 2-3 行为每个非空切面分配一个变量i来获取最外维度I的索引值。数组$slcIdx$和$slcPtr$存放张量所有非空切面索引值和内部非空纤维

中国知网 https://www.cnki.net

在数组$fbrIdx$中的位置，$slcPtr$中两个连续的值之间的纤维属于同一切面。第 4-5 行遍历当前切面所有的非空纤维，使用j获取纤维的索引值。数组$fbrIdx$和$fbrPtr$存放所有非空纤维的索引值和当前纤维包含的非零元在$innerFbrIdx$中的位置，$fbrPtr$中两个连续的值表示当前纤维包含的非零元。第 6-7 行遍历最后一个包含索引的数组，使用变量nnz访问非零元的值和在纤维中的相对索引。$fbrIdx$和$innerFbrIdx$这两个数组决定了参与计算的矩阵B和C元素。第 7 行和第 9 行根据公式(3.3)，分离了矩阵B和C与非零元的计算，将原来的单循环变成两层循环。

算法 3.3 基于 CSF 格式的 MTTKRP 算法

输入：张量的 CSF 格式 slcIdx[S], slcPtr[S], fbrIdx[F], fbrPtr[F],
innerFbrIdx[NNZ], vals[NNZ], Slice 个数 S, fiber 个数 F,
非零元个数 NNZ, 秩 R, 稠密矩阵 B[J][R],C[K][R]

输出：稠密矩阵 M[I][R]

```
1.  for r= 0 → R do
2.      for s= 0 → S do
3.          i = slcIdx[s]
4.          for f= slcPtr[s] → slcPtr[s + 1] do
5.              j = fbrIdx[f]
6.              for nnz = fbrPtr[f] → fbrPtr[f + 1] do
7.                  inB += vals[nnz] * B[innerFbrIdx[nnz]][r]
8.              end
9.              M[i][r] += inB * C[j][r]
10.         end
11.     end
12. end
13. return M
```

在 CSF 格式下分离非零元与两个因子矩阵的操作，张量的纤维与矩阵B的列进行内积计算之后再与矩阵C进行计算，此时纤维计算是独立的。同时张量的切面索引决定结果矩阵的行号，切面内部的计算不会对切面之间产生影响。因此考虑按切面对算法 3.3 使用 OpenMP 进行并行化，如算法 3.4 所示此时每个线程处理若干切面，不会发生读写冲突。第 3 行表示切面内的纤维的计算与算法 3.3 一致，线程对切面的划分不影响内部的纤维计算。

中国知网 https://www.cnki.net

算法 3.4 基于 CSF 格式的并行 MTTKRP 算法

输入：张量的 CSF 格式 slcIdx[S], slcPtr[S], fbrIdx[F], fbrPtr[F], innerFbrIdx[NNZ], vals[NNZ], Slice 个数 S, fiber 个数 F, 非零元个数 NNZ, 秩 R, 稠密矩阵 B[J][R],C[K][R]

输出：稠密矩阵 M[I][R]

```
1.  #pragma omp parallel for
2.  for s= 0 → S do
3.          i = slcIdx[s]
4.          for f= slcPtr[s] → slcPtr[s + 1] do
5.              j = fbrIdx[f]
6.              for nnz = fbrPtr[f] → fbrPtr[f + 1] do
7.                  inB += vals[nnz] * B[innerFbrIdx[nnz]][r]
8.              end
9.              M[i][r] += inB * C[j][r]
10.         end
11. end
12. return M
```

3.4 实验

3.4.1 实验环境

本节介绍实验使用的平台和数据集，并通过实验验证本文设计的不同压缩格式的并行 OpenMP 下的 MTTKRP 算法的有效性。本文使用的服务器 CPU 为 Intel Xeon Silver 4110，CPU 主频为 2.1Ghz，内存为 125G，共有 32 个 CPU，核心线程数为 2，编译器为 g++ 5.5.0（OpenMP 3.0）。

表 3.1 数据集信息

序号	张量	维度I	维度J	维度K	非零元数
1	ratings1	6040	3952	5	1000209
2	ratings10	71568	65134	5	10000054
3	ratings25	162542	209172	5	25000096
4	artist	2100	18744	12647	186479
5	nell-2	12092	9148	28818	76879419

中国知网 https://www.cnki.net

本文使用来自实际应用程序的稀疏张量，选取了 Never Ending Language Learning 数据集[53]（nell-2, noun-verb-noun），MovieLens 数据集[54]（ratings1, ratings10, ratings25, userID-movieID-ratings）和 Last.fm[55]数据集（artist, userID - artistID-tagID）。表 3.1 表示各个数据集的维度与非零元数量。

3.4.2 实验结果与分析

本文对 COO 格式和 CSF 格式的 MTTKRP 算法测试表 3.1 的数据集，设置张量的秩为 32，即矩阵的列为 32。MTTKRP 的性能用加速比来刻画，加速比定义为$s = t_1/t_2$，其中t_1表示串行算法的计算时间，t_2表示并行算法的计算时间。

得到的串行计算时间如表 3.2 和表 3.3 所示，明显看出 CSF 格式的算法优于 COO 格式。COO 格式下张量索引的访问是连续的，但是因子矩阵的访问无法有效利用缓存。尽管 CSF 格式对矩阵的访问是不连续的，但是公式(3.3)通过分离循环有效降低操作次数，显著减少了运算时间。

表 3.2 CSF 格式下的 MTTKRP 串行算法时间

数据集	Mode-1 的时间（秒）	Mode-2 的时间（秒）	Mode-3 的时间（秒）
ratings1	0.043784	0.046025	0.052736
ratings10	0.448892	0.798531	1.587033
ratings25	1.308587	2.984098	5.302444
artist	0.012755	0.023659	0.025679
nell-2	9.729368	9.286699	5.94559

表 3.3 COO 格式下的 MTTKRP 串行算法时间

数据集	Mode-1 的时间（秒）	Mode-2 的时间（秒）	Mode-3 的时间（秒）
ratings1	0.107007	0.072538	0.07186
ratings10	1.363921	1.152857	1.132471
ratings25	3.895328	3.492463	3.435276
artist	0.02153	0.021104	0.019588
nell-2	16.236555	14.157579	13.513376

图 3.2 和图 3.3 分别表示 CSF 格式和 COO 格式在不同线程下所有模态的平均加速比。可以看出线程并行的方式与串行相比，都得到了较高的加速比。在 CSF 格式的并行算法中，ratings10、ratings25 和 nell-2 这三个张量在线程数为 2~4 时，并行效率最高。artist 和 ratings1 在线程数为 32~64 之间时，并行效率最高。其中 artist 张量在线程数为 32 时，取得最高加速比 1.82。在 COO 格式的并行算法中

中国知网 https://www.cnki.net

artist 张量在线程数为 32 时加速效果最好，其余张量在线程数为 2~4 时加速效果最好。ratings1 在线程数为 4 时，并行加速比最高达到 3.67。尽管 COO 格式的并行算法整体加速比要高于 CSF 格式，但是计算时间都长于 CSF 格式的并行算法。

OpenMP 并行的 MTTKRP 算法的加速比达不到串行方法的线程倍数，原因首先在于张量的规模和张量的稀疏程度。非零元的索引决定了对矩阵的访问方式，这种访问方式往往是不连续的。张量的规模和稀疏程度越高，数据的访问越不规则，重复的内存访问就越多。同时线程的并行方式对加速比有很大的影响，在 CSF 格式的并行算法中张量的切面数决定了活跃线程的数量，在 COO 格式中则由张量的秩决定。

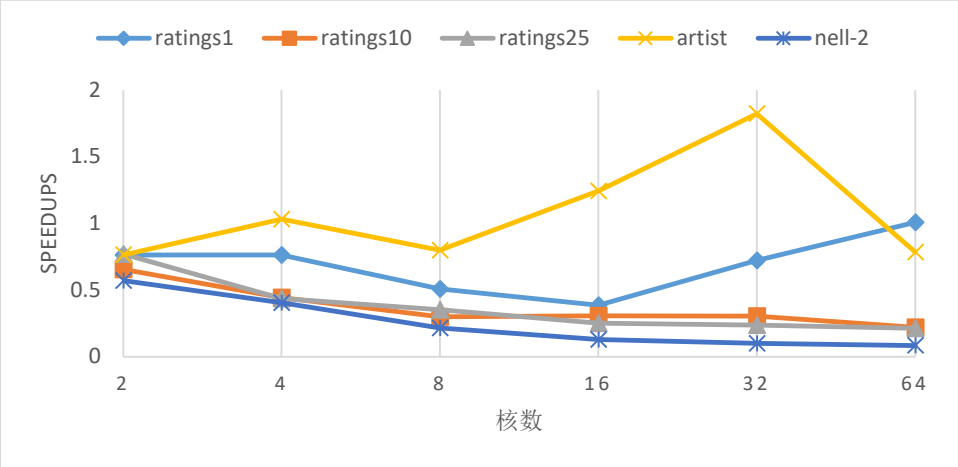


图 3.2 CSF 格式下的算法加速比

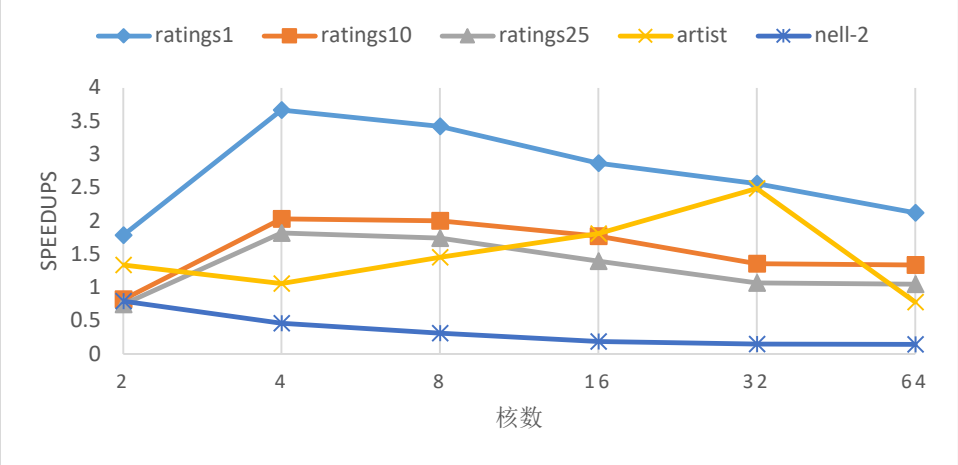


图 3.3 COO 格式下算法的加速比

3.5 本章小结

本章主要介绍了稀疏张量的存储格式，并在此基础上实现了 COO 格式和 CSF 格式的并行 MTTKRP 算法。COO 格式下各非零元计算相对独立，忽略非零元在纤维和切面上的潜在关联。使用 CSF 格式以纤维为单位利用纤维和切面索引进行计算能有效减少计算次数，纤维计算的并行程度更高。

中国知网 https://www.cnki.net

第 4 章 基于 GPU 的 MTTKRP 算法优化

GPU 并行计算的方式决定了其在大规模重复计算上的优势，用于处理相互无依赖关系和逻辑关系的大规模数据。GPU 上常用的并行级别为线程块之间的并行和线程之间的并行，多层次的并行可以灵活调节并行粒度。图 3.1 可知张量的 MTTKRP 运算包含大量的独立向量内积操作，且对张量进行划分得到的切面是相互独立的，因此 GPU 的并行方式可以有效提升算法的计算效率。由于张量的高维特性且张量维数决定因子矩阵维度，直接计算 Khatri-Rao 乘积会产生大量的中间变量，而张量的高稀疏性使得部分中间变量不会对结果产生影响。

为了充分说明本文在 GPU 上 MTTKRP 算法的优化，首先从一个简单的实现开始，逐步考虑 GPU 架构为 MTTKRP 提出三种并行优化方法。假设张量和密集矩阵 B 和 C 都已存储于 GPU 的显存中，其中张量采用 CSF 压缩格式。

4.1 并行算法基本实现

MTTKRP 在 CSF 压缩格式下根据 GPU 常用于矩阵乘法并行方式设计算法如算法 4.1 所示，每个线程负责计算结果矩阵的一个元素。GPU 使用一维网格和二维线程块，$threadIdx.x$指向张量的切面，$threadIdx.y$作为因子矩阵的列索引。

张量的稀疏性使得每个线程处理的切面中非零元的数量不同，可能会导致严重的线程负载不平衡。例如第 1 个切面有数以万计的非零元，而第 2 个切面的非零元数可能少于十个，负责计算这两个切面的线程很有可能处于同一个线程束中。这意味着受制于 GPU 上 SIMT 执行方式，此线程束中所有的线程的运算时间为负责最多非零元的切面中线程的计算时间。当前切面非零元较少时，线程计算完后需要等待线程束其余计算量较大的线程全部结束后，才能进行下一轮运算。分析算法流程，可以观察到如下缺点：

1) 线程束内的线程负载不均。每个线程都必须遍历张量中的一个切面和两个因子矩阵的一列。张量的稀疏性导致每个切面上的非零元数量的不均衡，因此执行时间短的线程必须等待线程束中其它线程，这样的线程分配方式是粗粒度的。
2) 非合并内存访问。纤维内部的索引值和因子矩阵的列索引决定因子矩阵的访存方式，这两个变量会产生大量重复无用的内存事务。
3) 片上内存利用率低。算法 4.1 只访问全局内存，没有利用速度更快的片上内存。每个线程都需要从全局内存中独立访问完整的切面。可以将数据加载到共享内存中，以共重复访问的数据使用。

中国知网 https://www.cnki.net

算法 4.1 CSF-MTTKRP 的核函数基本实现

输入：张量的 CSF 格式 slcIdx[S], slcPtr[S], fbrIdx[F], fbrPtr[F], innerFbrIdx[NNZ], vals[NNZ], 切面个数 slcNum，秩 rank，GPU 线程层次 dimGrid 和 dimBlock, 稠密矩阵 B[J][R], C[K][R]

输出：稠密矩阵 M[I][R]

```
1.  m = threadIdx.x + blockIdx.x * blockDim.x
2.  f = threadIdx.y + blockIdx.y * blockDim.y
3.  if m < slcNum and f < rank then
4.      innerC = 0
5.      for i = slcPtr[m] → slcPtr[m+1] do
6.          innerB=0
7.          for j = fbrPtr[i] → fbrPtr[i+1] do
8.              innerB += vals[j] * B[innerFbrIdx[j]][f]
9.          end
10.         innerC += innerB * C[fbrIdx[i]][f]
11.     end
12.     M[slcIdx[m]][f] = innerC
13. end
```

4.2 并行算法优化方式

4.2.1 切面并行

切面维度作为张量的最外围维度，公式(3.2)表明 MTTKRP 运算中切面之间是无关的，且在计算中主要使用两个用于表示索引和位置的数组$slcIdx$和$slcPtr$。在 GPU 的计算中线程块是网格内的第一层并行方式，具有表示索引的内置变量$blockIdx$。因此可以给每个切面分配若干线程块，其数量由设定的张量的秩决定。切面的索引值不需要从 CPU 传到 GPU 上，$blockIdx$作为数组$slcPtr$的索引确定当前切面的非空纤维，减少 CPU 与 GPU 之间的数据传输和对全局内存的访问。考虑到对矩阵的遍历，如图 4.1 可以将算法 4.1 中的一维网格扩展为二维网格，网格的维度根据张量的非空面和矩阵的列数设置为

$$dimGrid = (rank / blockDim.x, slcNum)$$

线程块的维度设置成 32 的倍数，$blockIdx.y$指向张量的切面即结果矩阵的行，$blockIdx.x$用来确定矩阵的列索引。

中国知网 https://www.cnki.net

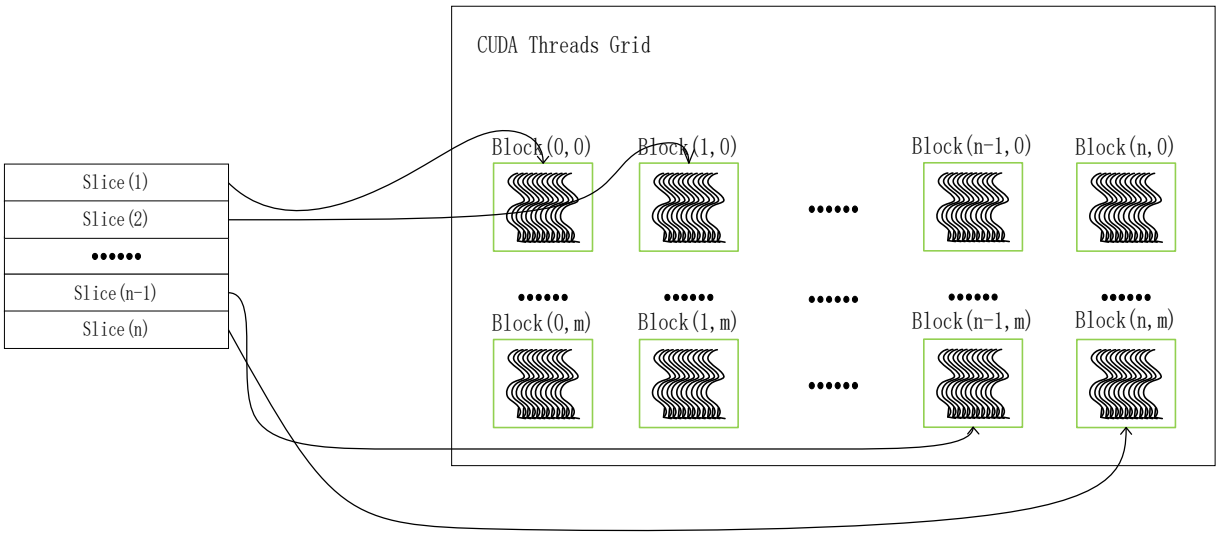


图 4.1 切面的划分方式

4.2.2 合并访存

CSF 格式的张量在使用切面并行方式后，仍需要在全局内存中存储五个数组。全局内存作为 GPU 上空间最大延迟最高的内存，常通过合并访存来最大化全局内存带宽。合并的内存访问是指将多个内存访问组合到单个事务中。张量中的一个切面与两个因子矩阵的计算得到结果矩阵的一行，结果矩阵行中的每个元素都可以视为对应切面的非零元和矩阵列的乘积之和。每个非零元必须与完整的因子矩阵的一行相乘，这意味着对矩阵的访问是连续的。

算法 4.1 中观察到，张量的索引和值是连续的，而对矩阵的访问是非连续的。矩阵 B 的行索引$innerFbrIdx[j]$和矩阵 C 的行索引$fbrIdx[i]$不同，线程束中每个线程访问的矩阵地址是不连续的，产生了非合并访存。为了避免线程束对矩阵行的随机访问，令维度x指向矩阵的列，令维度y指向切面的纤维。此时遍历线程的索引x即为遍历矩阵的一行，对矩阵的访问总是合并的。

在 GPU 上进行张量分解时，张量的秩常设置为线程束的大小，因此本文设张量的秩为 32，即矩阵的列数为 32。采用二维线程块，$threadIdx.y$即为线程束在当前线程块中的索引。线程束中所有线程依次访问连续的内存块，即矩阵B和C的一行。纤维的计算由线程束负责，非零元每次与矩阵行的所有元素进行乘法操作。观察公式(3.3)可以发现张量的纤维与矩阵列向量的内积之间是无依赖关系的，因此可以考虑关于矩阵的列向量的并行方式。

线程束可以同时处理纤维与矩阵所有的列向量的内积，线程执行方式如图 4.2 所示。此时位于这个线程束中所有的线程操作不会产生数据依赖，且所有线程同时结束不发生线程空等或假执行。调整线程块维度后，线程束由原本的同时处理张量的切面变为同时处理张量的纤维，显著减小了任务粒度。计算纤维的非零元

中国知网 https://www.cnki.net

与矩阵行的乘法时，线程束中的线程是独立进行的。同一个线程束的线程在同一周期执行相同的操作，同时访问矩阵行的所有元素。此时内存事务的所有数据都被利用，不会发生线程束分化的情况。假设一个线程束分得m个纤维，那么线程$(threadIdx.x, threadIdx.y)$负责处理

$$innerC = \sum_{k=0}^{m} C(k,f) \sum_{j=0}^{J} \mathcal{X}(i,j,k)B(j,f) \tag{4.1}$$

其中

$$f = threadIdx.x + blockIdx.x \times blockDim.x$$

每个线程束处理若干纤维，每个线程求得当前切面对应行的结果分量，所有线程束的结果进行归约求和得到完整行数据。

调整线程块的维度x执行方式，可以减少对全局内存的访问延迟。维度x指向矩阵的行将算法 4.1 中对矩阵行的随机访问转换成对行的合并访存，提高内存事务的利用率。同时，设计线程执行方式使得线程束每次访问矩阵行的所有元素，每次执行一个非零元与矩阵行的乘法，避免线程束分化。

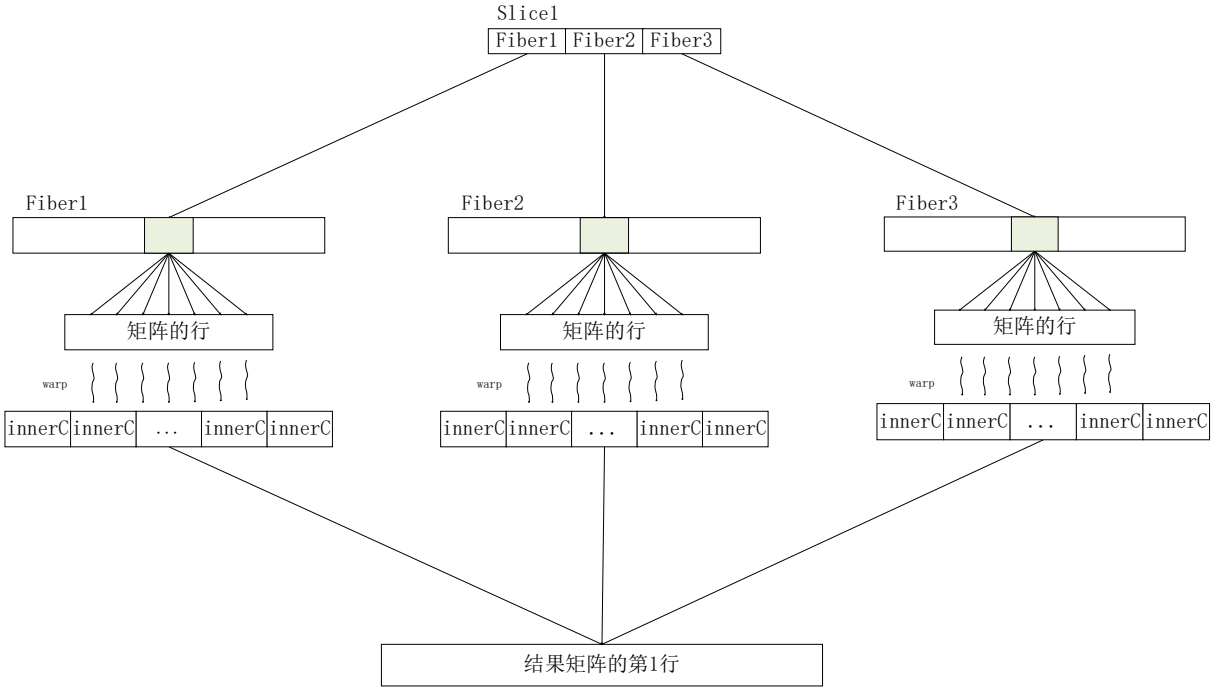


图 4.2 线程块中线程的操作

4.2.3 共享内存

共享内存是片上内存，它的访存延迟仅次于寄存器，比大多数显存快很多。共享内存的生存周期与线程块保持一致，同一个线程块上的线程可以访问其它线程加载到共享内存的数据，实现高效并行。

观察图 4.2 可以看出，所有线程束在计算完非零元后需要相互通信，进行求

中国知网 https://www.cnki.net

和之后得到最终结果。在张量各切面的计算是无关的前提下，线程块内的线程束都是在同一个切面内进行，而共享内存是线程块特有的片上内存且支持线程束通信，因此本文使用共享内存来存储线程块内每个线程束的结果。

每个线程束产生32个结果分量$innerC$，每个$innerC$为结果矩阵在第$blockIdx.x$行第f列上的分量。假设一个线程块使用n个线程束，那么存储这些分量可以使用大小为$n \times 32$的矩阵，即线程块的线程数量，如图 4.3 为$n = 4$的共享内存。在线程块中创建大小与块内线程数一致的二维共享矩阵shr，存储每个线程的$innerC$。一个线程束分得二维共享矩阵的一行，线程束中每个线程的结果$innerC$都在共享内存不同的存储体上，此时对共享内存的访问只需要一个内存事务。现有一线程在其线程块中的序号为(thx, thy)，将该线程的$innerC$存入共享内存中，即

$$shr[thy][thx] = innerC$$

结合公式(4.1),对每列进行求和操作得

$$M[blockIdx.y][f] = \sum_{i} shr[i][j]$$

即为结果矩阵对应行的元素。

本文采用并行归约算法，对矩阵的列进行求和。此时所有线程束中相同序号的线程计算的$innerC$都在同一个存储体上。这样的存储方式使得数据的加载到共享内存只需要一个内存事务，但是求和操作都是在矩阵的列中进行，即多个线程访问同一个存储体的不同地址，这会造成严重的存储体冲突，降低共享内存的带宽。为了避免存储体冲突的发生，本文使用内存填充操作将二维共享矩阵大小调整为$blockDim.y \times (blockDim.x + 1)$，填充的数据将原来位于同一存储体的数据传播到不同的存储体中。按行将$innerC$存储到共享内存，线程束中各线程访问不同的存储体。在按列进行求和时，由于打乱了数据在列中的排列，逻辑上列数据并不在同一个存储体中。在不同的存储体上对数据进行归约求和，最大限度地提高线程束中的并发访问数量。

Warp1	innerC	innerC	...	innerC	innerC
Warp2	innerC	innerC	...	innerC	innerC
Warp3	innerC	innerC	...	innerC	innerC
Warp4	innerC	innerC	...	innerC	innerC

图 4.3 线程束产生的共享矩阵

在求$M[blockIdx.y][f]$的过程中，使用交错配对的方式进行归约求和。图 4.4 表示在共享矩阵shr上的列使用交错归约，并将结果放到矩阵的第一行中。交错归约在每次循环中元素跨度减少一半，初始跨度为当前线程束的一半。线程每次访

中国知网 https://www.cnki.net

问当前线程号所指的存储体与被当前跨度隔开的另一个存储体，交错归约使得线程束中所有的线程不会同时访问相同的存储体。

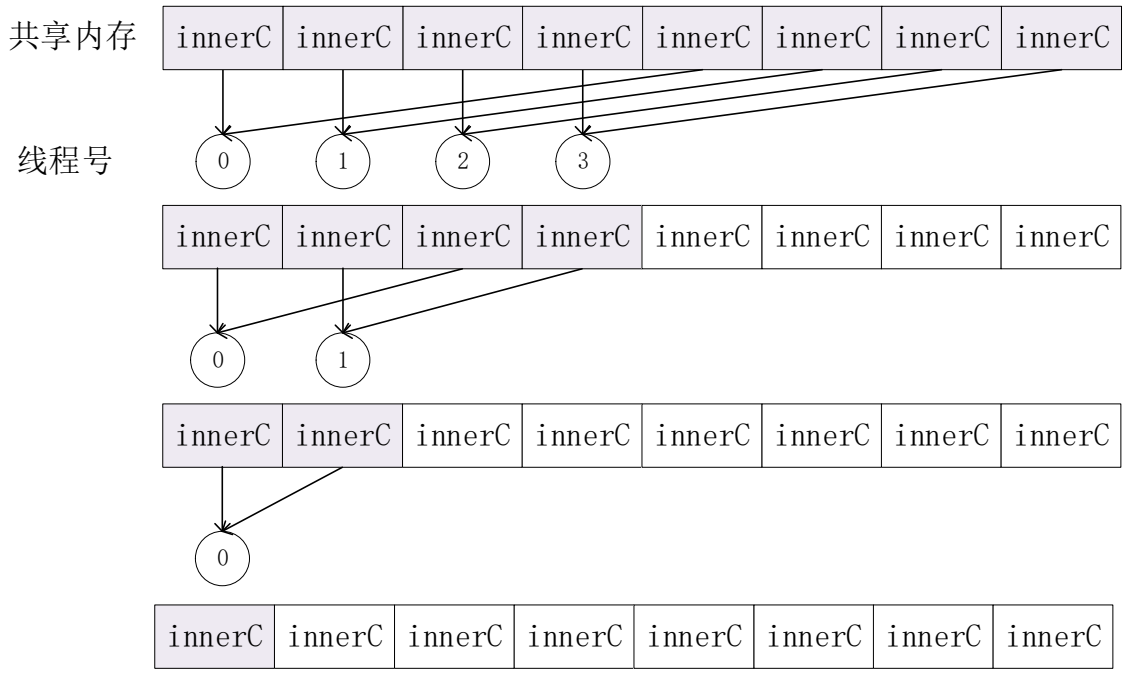


图 4.4 共享矩阵按行归约求和

4.3 优化并行算法的实现

本文从 COO 格式下的 MTTKRP 算法入手，观察算法 3.1 可以发现 COO 格式没有利用非零元索引值之间的关系，忽略了张量切面与纤维对算法的影响。COO 格式是一种独立性强的存储格式，同一个切面或者同一个纤维上的非零元在 COO 格式中是无关的。切面的序号关系到结果矩阵的行号，而纤维索引确定参与计算的矩阵C的行索引，非零元在纤维内的相对索引确定矩阵B的行索引。CSF 格式作为一种分层的以纤维为中心的压缩格式，有效提高了切面与纤维的索引重用率。

根据 CSF 格式，结合 GPU 设计使用二维线程块的并行算法，根据张量的切面数量和矩阵的列划分线程，一个线程负责计算结果矩阵的一个元素。这种算法提高了非零元索引的利用率，但没有考虑 GPU 的存储特点。线程必须遍历张量的切面和两个因子矩阵的一列。而张量的稀疏性使得切面之间的非零元数量不均衡。由于 GPU 是按照 SIMT 的方式执行，线程束的执行时间取决于其中计算非零元最多的线程，负责数据少的线程只能空等或假执行。两个因子矩阵的访问是随机不规律的，这样的访问模式会产生大量重复且无用内存事务。所有的数据都是直接访问访存延迟最高的全局内存，片上内存的利用率低。因此，本文从调整并行粒度，增加合并访存，设计并行方式，使用共享内存出发，基于 GPU 优化 CSF 格式下的稀疏张量的 MTTKRP 算法，称为 Opt-MTTKRP 算法。

中国知网 https://www.cnki.net

算法 4.2 Opt-MTTKRP 的核函数基本实现

输入：张量的 CSF 格式 slcPtr[S], fbrIdx[F], fbrPtr[F],
innerFbrIdx[NNZ], vals[NNZ], 切面个数 slcNum，
GPU 线程层次 dimGrid 和 dimBlock, 稠密矩阵 B[J][R], C[K][R]
输出：稠密矩阵 M[I][R]

```
1.  Set bdx=blockDim.x, bdy=blockDim.y, thx=threadIdx.x, thy=threadIdx.y,
      bIdx=blockIdx.x, bIdy=blockIdx.y, f=thx+bIdx*bdx,
      fbrNum=slcPtr[bIdy+1]-slcPtr[bIdy]
2.  Allocate shared matrix shr[bdy][bdx+1]
3.  if f<rank then
4.      firstFbr=slcPtr[bIdy]
5.      for i = thy → fbrNum by i += bdy do
6.          for j=fbrPtr[firstFbr + i] → fbrPtr[firstFbr + i + 1] do
7.              innerB += vals[j] * B[innerFbrIdx[j]][f]
8.          end
9.          innerC += innerB * C[fbrIdx[firstFbr+i]][f]
10.     end
11.     shr[thy][thx] = innerC
12.     for stride = bdy/2 → by stride >>= 1 do
13.         if thy < stride then
14.             shr[thy][thx] += shr[thy+stride][thx]
15.             __syncthreads()
16.         end
17.     end
18.     if thy == 0 then M[bIdxy][f] = shr[0][thx]
19. end
```

算法 4.2 中给出在线程划分为$dimGrid = (rank / blockDim.x, slcNum)$和$dimBlock = (32,16)$下的核函数伪代码。本文结合 GPU 的并行层次与张量的分层存储模式来提高 MTTKRP 运算，对任务进行划分，合理调整并行粒度；改变张量的压缩格式，提高数据的重用，以节约 GPU 的显存空间；优化运算步骤，使用共享内存减少中间变量的使用和计算的操作数。

张量的切面之间的数据是无关的，可以通过调整线程的分块方式，根据矩阵的列数和张量的切面数划分线程块。这种划分方式不需要 CSF 格式中$slcIdx$数组的传输，减少了 GPU 中的全局内存的数据量。

中国知网 https://www.cnki.net

合并访存作为减少全局内存访问延迟的重要方法，可以用最少的事务次数满足最多的内存请求。张量的切面与两个因子矩阵的计算得到结果矩阵的一行，切面中每个非零元与完整的因子矩阵行相乘，此时对矩阵的访问是连续的。因此，令二维线程块内的维度x指向矩阵的列，维度y指向切面的纤维，此时对矩阵的访问是按行的合并访存。

使用 GPU 做张量计算时，张量的秩通常设置为 32，$threadIdx.y$可以看作是线程束在线程块中的序号。线程束分配切面中的纤维，线程负责非零元与矩阵行的所有元素进行乘法操作。张量的纤维与矩阵列向量的内积之间是独立的，本文设计关于矩阵的列向量的并行方式。线程束同时处理纤维与矩阵所有列向量的内积，所有线程同时结束不会发生空等或假执行。线程束的所有线程同时访问矩阵行的所有元素，此时只需要一个内存事务。每个线程束处理若干纤维，每个线程求得当前切面对应行的结果分量$innerC$，所有线程束的结果进行归约求和得到完整行数据。

共享内存作为访存较快的片上内存，可以被线程块上的所有线程共享。设置与线程块大小一致的二维共享矩阵，每行数据存储每个线程束的结果分量，达到线程块内线程对数据高速访问的目的。在共享矩阵上对每列进行交错归约求和，求和结果即为当前线程块对应结果矩阵的一行。

4.4 实验

4.4.1 实验环境

本节介绍实验使用的平台，并将本文的 Opt-MTTKRP 算法与现有的基于 GPU 的 MTTKRP 算法进行比较分析。设张量的秩为$R = 32$，使用 32 位无符号整数存储索引值，32 位单浮点数存储非零值。

本文使用的设备 GPU 为 NVIDIA TITAN RTX，包含 72 个流多处理器，显存为 24GB，L2 缓存为 6MB，核心频率为 1770MHz，单精度性能最高可达 16.31 TFLOPS。代码使用 g++ 5.5.0 和 NVCC 10.1 进行编译。

本文通过 ParTI[56]，B-CSF 和 HB-CSF[57]提供的算法进行性能对比实验，ParTI 注重优化稀疏张量的 MTTKRP 算法在 GPU 上的计算方式，B-CSF 和 HB-CSF 通过改进 CSF 的压缩格式来提高 GPU 的吞吐量。

ParTI 在 GPU 上设计了就地转化的 COO-MTTKRP 算法来避免张量矩阵的数据转换，同时使用辅助数组并行算法的读写冲突。针对单节点 GPU 上 MTTKRP 算法采用精细的线程粒度，安排合并内存访问和张量的秩分块。对于 COO-MTTKRP，很难利用 GPU 共享内存的优势。所有实现都使用 CUDA 的原子操作来解决数据竞争问题。

中国知网 https://www.cnki.net

Balanced-CSF (B-CSF) ，作为 CSF 格式的扩展将负载较重的切面划分成若干子切面。这种方式可能会增加分配到同一个切面的线程块中的原子操作数量。但是并发性的增加可以容忍额外的原子操作的成本。考虑到纤维的非零元个数，负载中的纤维可能比其它纤维需要更长的处理时间，从而使闲置的线程束停滞在线程块中。将长的纤维拆分成纤维段，这使得线程库中的所有线程束的负载几乎相等。B-CSF 在构造 CSF 格式时完成此预处理步骤，从而避免了任何额外的预处理时间。

Hybrid-CSF (HB-CSF) 从 CSF 格式对稀疏张量分配不均出发，为张量开发了一种新的存储高效表现形式，可以在 GPU 上实现高性能负载均衡的 MTTKRP 算法。HB-CSF 被设计成包含 COO，CSF，CSL (Compressed Slice) 的混合格式，单个非零元的切面直接用 COO 格式存储。CSL 格式针对每个纤维只有一个非零元，即单纤维的切面，只使用四个数组来存储索引数据，分别为切面索引，切面指针以及其余两个维度的索引。切面指针可以直接指向切面中的非零元，避免通过纤维指针进行间接访问。同时减少了纤维的非零元的局部归约。其余切面采取 CSF 格式。

本文假设计算之前数据已经存储在 GPU 的显存中，不计算 CPU 与 GPU 的传输时间。MTTKRP 的性能用加速比来刻画，加速比定义为$s = t_1/t_2$，其中t_1表示对比算法的计算时间，t_2表示 Opt-MTTKRP 算法的计算时间。

4.4.2 实验结果与分析

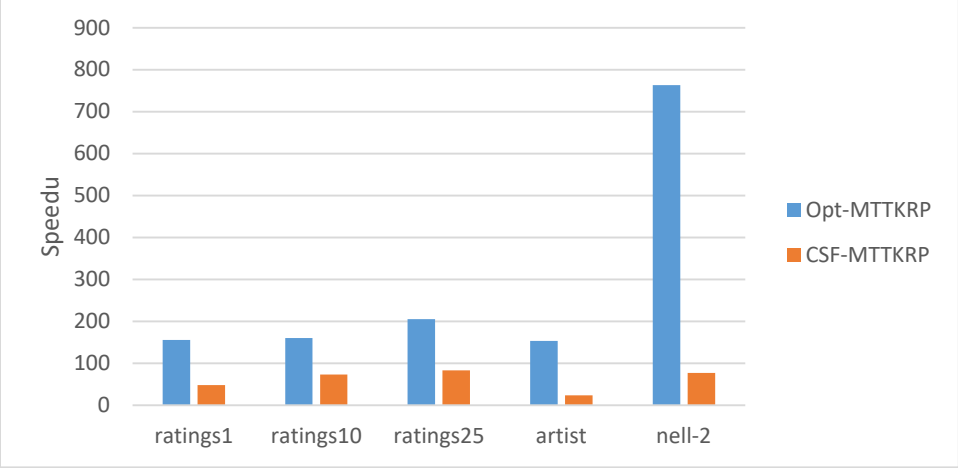


图 4.5 Opt-MTTKRP 和 CSF-MTTKRP 的加速比

本文在单节点 GPU 平台上利用表 3.1 所示数据集测试算法，图 4.5 表示在 GPU 上 CSF-MTTKRP 算法和本文的 Opt-MTTKRP 算法相对于基于 CSF 格式的串行 MTTKRP 算法的加速比。所有的加速比都是在 mode-1 上的对 MTTKRP 的五次迭代取平均值得出，GPU 上线程块的大小设置为$dimBlock = (32,16)$，线程块的数量取决于张量的切面数。经过优化的 Opt-MTTKRP 算法在五个真实张量上的

中国知网 https://www.cnki.net

加速比最高可达 763.17 倍，而 CSF-MTTKRP 算法加速比最高可达 83.31 倍，这充分体现出 GPU 在密集计算上的优势。

三维张量有三个模态，图 4.6、图 4.7 和图 4.8 分别表示 Opt-MTTKRP 算法基于 ParTI-COO 算法，B-CSF 算法和 HB-CSF 算法下在 GPU 上的各个张量的模态的加速比。

在图 4.6 中，Opt-MTTKRP 算法的平均性能是 ParTI-COO 算法的 1.60 倍，artist 数据的 mode-1 下达到了最高的加速比 3.58 倍。Opt-MTTKRP 算法中线程块数取决于当前的模态，处于活跃状态的线程块受到 GPU 资源（如寄存器和共享内存）的限制。实际上在 mode-1 下 artist 张量有 1892 个切面，35816 个纤维和 186479 个非零元，导致更高的数据局部性和更多活跃状态的线程块。在 mode-2 和 mode-3 下，COO 格式明显要要优于 CSF 格式，此时纤维之间元素分布不均衡，使用 COO 格式占用的空间更低。

图 4.7 比较 Opt-MTTKRP 算法与 B-CSF 算法，可以得到 Opt-MTTKRP 的平均加速比为 1.87 倍。与 PatTI-COO 相同的是都是在 artist 张量达到上达到最高加速比，不同的是 rantings1，rantings10 和 ratings25 在 mode-3 下加速效果不明显。由于张量的维度K仅为 5，因此在 mode-2 中单个 warp 的负载较大，在 mode-3 中分配的线程块过少，这些因素都使得 Opt-MTTKRP 的加速效果较弱。

图 4.8 可以得到 Opt-MTTKRP 算法与 HB-CSF 的比较结果，结果与 B-CSF 类似，Opt-MTTKRP 的平均加速比为 1.81 倍。HB-CSF 是在对张量进行划分的基础上，使用混合格式进行计算。对于切面不满足划分标准的张量，这种混合格式的加速效果较弱。

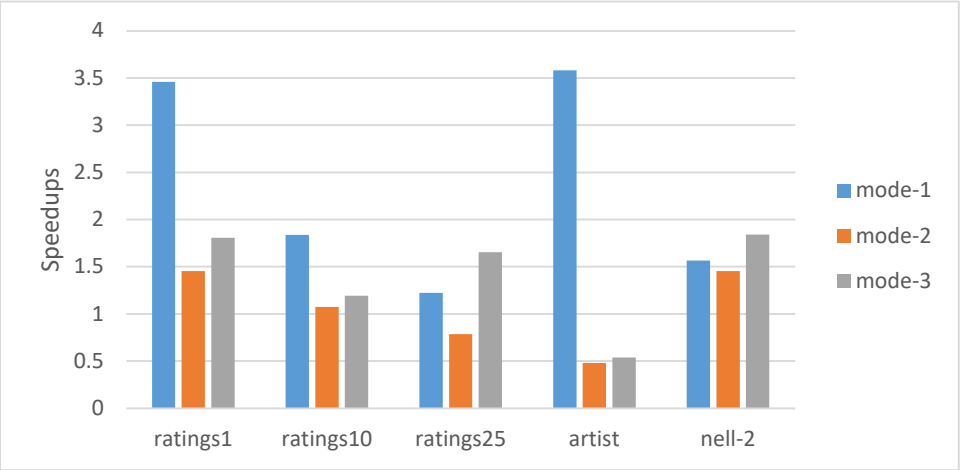


图 4.6 基于 ParTI-COO 的加速比

中国知网 https://www.cnki.net

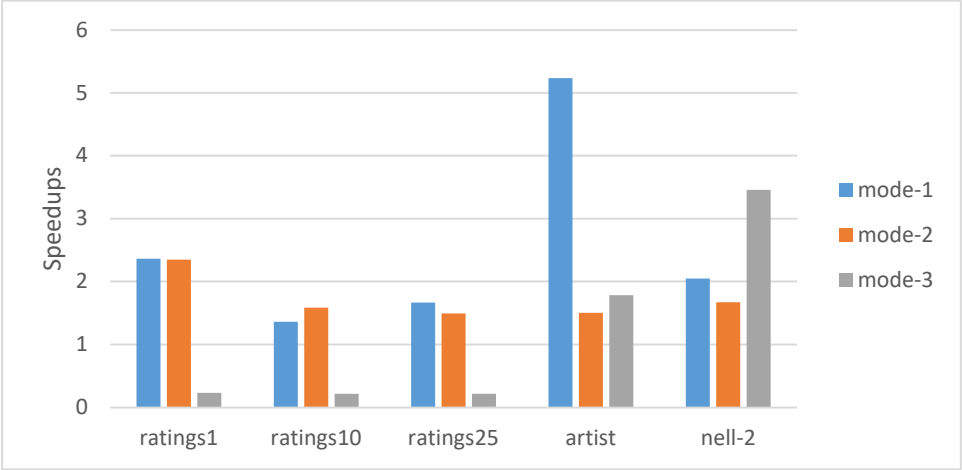


图 4.7 基于 B-CSF 的加速比

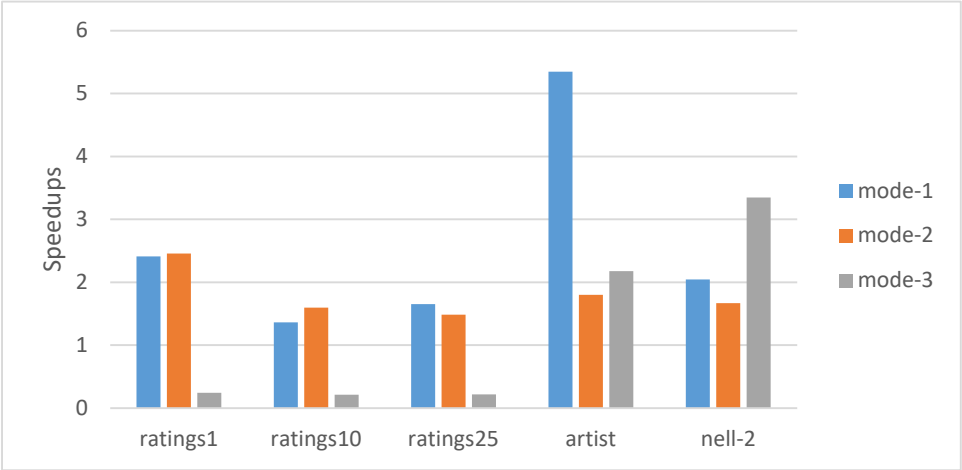


图 4.8 基于 HB-CSF 的加速比

本文提出了三中优化策略，分别是切面级并行，合并内存访问和共享内存。切面级并行减少主机和设备之间的传输，合并内存访问提高内存事务的利用率，共享内存用于在同一线程块中的线程交换数据。

图 4.9 表示三种策略对 CSF-MTTKRP 的加速效果，Naïve 表示 CSF-MTTKRP，CA 表示合并访存，SS 表示切面级并行和共享内存两种策略结合使用以及 SSC 表示 Opt-MTTKRP。

定义策略i的性能提升率r为

$$r = \frac{S_i - \hat{S}}{\hat{S}}$$

其中$\hat{S}$表示 CSF-MTTKRP 算法在所有张量的所有模态下的平均加速比，S_i表示策略i所有模态下的平均加速比。所有的优化策略都使得 CSF-MTTKRP 性能有所提高，合并访存使得性能提高 234%，切面级并行与共享内存合并使用性能提高 38%，所有的优化策略可以同时提高性能 403%。由于 nell-2 张量中纤维之间非零元分布均匀，各个纤维束负载更为均衡，因此 nell-2 中优化效果达到最好。优化策略中合并访存的加速效果最为明显，而切面级并行和共享内存两种策略易受到张量

中国知网 https://www.cnki.net

中各维度的不均衡的影响。

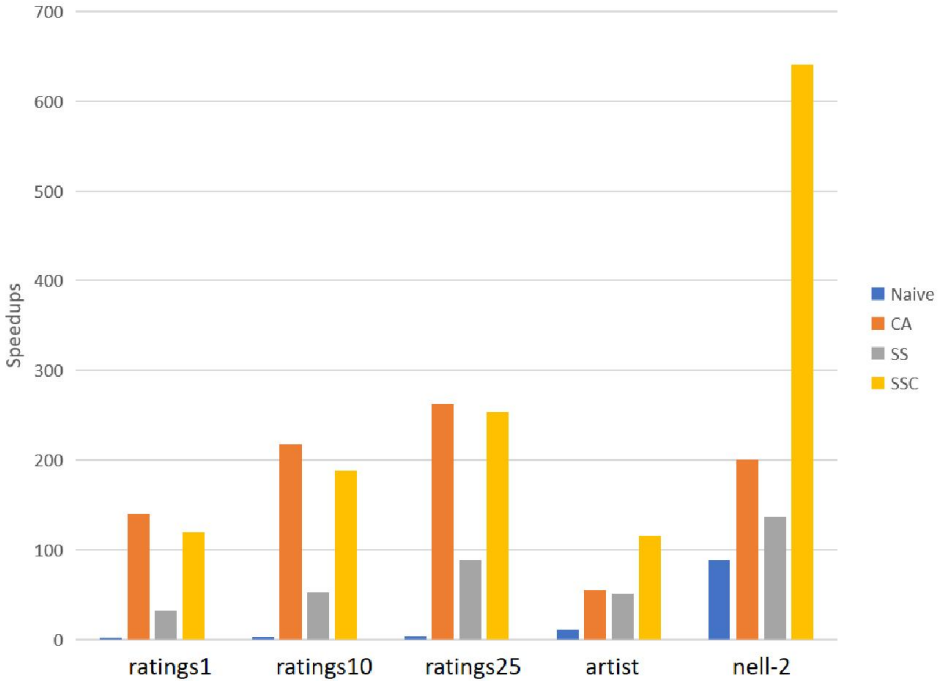


图 4.9 优化策略对性能的影响

4.5 本章小结

本章主要内容是提出了一种结合 GPU 的并行 MTTKRP 算法，称为 Opt-MTTKRP 算法，通过现实数据集上的验证，证明了本文的 Opt-MTTKRP 算法能有效加速 MTTKRP 计算，并且其性能在本文的数据集上优于现有算法。

中国知网 https://www.cnki.net

结论

信息技术的快速发展使得各个领域的数据收集、存储和分析的能力得到极大的提高，数据呈现规模大、类型复杂和分布不均等特点。使用传统的基于矩阵理论的数据分析方法分析高维数据不能完全反映出数据的复杂结构和潜在关联。张量作为高维数据的表现形式在大数据环境下已经成为学者们关注的重点。张量分解与主成分分析类似，将高维空间映射到低维空间，分解得到的低维数据保留高维的信息和结构。因此张量分解成为当前处理高维数据的一个热门方法。

高维数据在采集的过程中会产生大量空值，张量往往非常稀疏。数据分布会影响张量算法的性能，稀疏数据会产生大量空操作。在高维稀疏数据下张量运算遇到许多不可避免的计算问题，中间内存过大以及算法本身的高复杂度，这些都限制了张量的计算性能的提高。MTTKRP 计算作为常用的张量运算，一直是张量分解的计算瓶颈。CP 分解作为张量分解算法中的经典方法，将张量分解若干秩一张量，其 ALS 算法中 MTTKRP 计算会占用大量的内存，Khatri-Rao 积的数据量甚至会超过张量本身。本文分析不同压缩格式下的 MTTKRP 算法，从提高算法并行程度入手，优化 MTTKRP 算法，进行如下的探索：

1. 首先阐述张量和 MTTKRP 的研究背景意义，介绍现有研究学者的对张量分解中的 MTTKRP 改进方式。阐述了 CUDA 编程模型和存储模型，介绍 GPU 执行步骤和编程语法，说明高效编程中的线程束分化，非合并访存和存储体冲突等挑战。介绍了张量的基本概念和基础运算。给出了张量模态 n 的纤维和切面的定义，并在此概念上引入了张量矩阵化，张量矩阵乘法以及张量运算中常见的矩阵相关乘法。张量分解的目的是将原始张量分解为若干子张量，以提炼出张量潜在的模式和因子。着重介绍了 CP 分解，作为奇异值分解的扩展，将张量分解为若干个秩一张量。分析得到 MTTKRP 计算是 CP 分解的 ALS 算法中的计算瓶颈，为后文实现 MTTKRP 并行化提供基础。

2. 针对现实数据的稀疏性，介绍了张量不同的压缩格式，并设计 CPU 下不同格式的基于 OpenMP 的 MTTKRP 并行实现算法。分析 COO 格式下的 MTTKRP 的计算方式，发现 COO 格式无法有效利用张量非零元之间关于纤维和切面的潜在关联以及非零元与矩阵的索引关系。使用分层结构的 CSF 格式进行 MTTKRP 运算，分离两个因子矩阵与非零元的计算，有效减少计算操作数。此时纤维与矩阵列的计算是独立的，且切面内部的计算不影响切面之间的关联。通过现实数据集验证 CPU 并行算法，明显发现 MTTKRP 计算中 CSF 格式优于 COO 格式。

3. 针对 GPU 并行方式，提出了一种更高效的 MTTKRP 算法。选择分层结构的 CSF 格式存储稀疏张量，以 CSF 格式的串行算法为基础，提高算法并行程度

中国知网 https://www.cnki.net

为目的，实现了并行的 Opt-MTTKRP 算法。利用 GPU 中线程块的内置索引，实现算法的切面级并行，减少 CPU 到 GPU 的数据传输。调整线程块的线程执行方式，每个线程束对应因子矩阵的一行，实现对矩阵数据的对齐合并访存。线程束每次执行一个非零元与矩阵行的乘法，避免线程束分化。利用 CSF 格式下纤维运算的独立性，线程束分别计算纤维与矩阵的乘积。将每个线程束计算的结果分量存储到访存延迟低的共享内存上，通过内存填充避免存储体冲突，对共享矩阵进行按列求和得到结果矩阵对应行值。最后通过现实数据集验证了本文给出的并行算法的有效性，发现合并访存对性能提供最明显。

本文给出了 CPU 和 GPU 的并行 MTTKRP 算法，并分析不同压缩格式对算法的影响，能有效提高计算效率。在 GPU 计算过程中，本文只实现了线程束的线程负载均衡，线程块之间的负载均衡还需进一步对切面进行划分来达到 GPU 的负载均衡。切面级并行利用线程块号作为切面索引来减少 CPU 和 GPU 的数据传输量，这种划分方式受制于张量的切面数量。当张量的非零元分布在少数的切面上时，GPU 启用的线程块较少且每个线程块的负载较重。这样的并行方式无法有效利用到 GPU 的全部计算资源。论文仍存在很多不足与进步的空间，因此下一步的工作可以总结为：

1. 针对切面非零元不均的问题，采取切面划分的方式。设计一种自适应的分割算法，将非零元较多的切面分割成若干个子切面以达到子切面的非零元数目相近。切面级并行在自适应分割算法下，线程块可以达到负载均衡。对于切面数较少的稀疏张量，分割算法可以启动更多的活跃线程块同时线程等待数据的时间更少。线程束之间的计算时间差异降低，GPU 的流水线时间更短。

2. 针对 CPU 和 GPU 并行算法，设计异构协同算法。由于 CPU 和 GPU 是异步并行的关系，因此可以考虑对切面进行分配以实现 CPU 和 GPU 协同处理。根据 CPU 和 GPU 的计算能力分配切面，数据传输时间与 GPU 的计算时间达到和 CPU 的计算时间保持一致。

3. 针对 MTTKRP 算法的应用场景，设计高效的 CP 分解算法。不同于静态张量分解，增量式张量分解的结果总是随着张量的扩展不断改变，但是部分中间计算结果是可重用的。在增量式张量分解的情形下，保存每个模态常用的中间变量，优化每个因子矩阵的迭代过程。存储每个模态的张量压缩格式或者使用统一模态的压缩格式，提高数据的利用率。

对张量应用要求随着技术的发展会越来越多，高维度和高稀疏性已然成为张量分解的难点所在，如何合理地降低张量计算中的 MTTKRP 运算的计算成本需要进一步研究。

中国知网 https://www.cnki.net

参考文献

[1] Cherry S. Some comments on singular value decomposition analysis. Journal of Climate, 1997, 10(7):1759–1761.

[2] Du S, Shi Y, Hu W, et al. Robust Tensor Factorization for Color Image and Grayscale Video Recovery. IEEE Access, 2020, 8:174410–174423.

[3] Bengua J A, Phien H N, Tuan H D, et al. Efficient tensor completion for color image and video recovery: Low-rank tensor train. IEEE Transactions on Image Processing, 2017, 26(5):2466–2479.

[4] Yap P T, Wu G, Zhu H, et al. TIMER: Tensor image morphing for elastic registration. NeuroImage, 2009, 47(2):549–563.

[5] Wang H, Ahuja N. Rank-R approximation of tensors using image-as-matrix representation. In: Proc of 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), volume 2. IEEE, 2005, 346–353.

[6] Cichocki A, Mandic D, De Lathauwer L, et al. Tensor decompositions for signal processing applications: From two-way to multiway component analysis. IEEE signal processing magazine, 2015, 32(2):145–163.

[7] Sørensen M, De Lathauwer L. Tensor decompositions with Vandermonde factor and applications in signal processing. In: Proc of 2012 Conference Record of the Forty Sixth Asilomar Conference on Signals, Systems and Computers (ASILOMAR). IEEE, 2012, 890–894.

[8] De Almeida A. Tensor modeling and signal processing for wireless communication systems. Université de Nice Sophia Antipolis, 2007.

[9] Comon P. Tensor diagonalization, a useful tool in signal processing. IFAC Proceedings Volumes, 1994, 27(8):77–82.

[10] Huang X, Qiao H, Zhang B, et al. Supervised polarimetric SAR image classification using tensor local discriminant embedding. IEEE Transactions on Image Processing, 2018, 27(6):2966–2979.

[11] Newman E, Kilmer M, Horesh L. Image classification using local tensor singular value decompositions. In: Proc of 2017 IEEE 7th International Workshop on Computational Advances in Multi-Sensor Adaptive Processing (CAMSAP). IEEE, 2017, 1–5.

中国知网 https://www.cnki.net

[12] Shashua A, Levin A. Linear image coding for regression and classification using the tensor-rank principle. In: Proc of Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001, volume 1. IEEE, 2001, I–I.

[13] Zhong Z, Fan B, Duan J, et al. Discriminant tensor spectral–spatial feature extraction for hyperspectral image classification. IEEE Geoscience and Remote Sensing Letters, 2014, 12(5):1028–1032.

[14] Wang S J, Yang J, Zhang N, et al. Tensor discriminant color space for face recognition. IEEE Transactions on Image Processing, 2011, 20(9):2490–2501.

[15] Hua G, Viola P A, Drucker S M. Face recognition using discriminatively trained orthogonal rank one tensor projections. In: Proc of 2007 IEEE Conference on Computer Vision and Pattern Recognition. IEEE, 2007, 1–8.

[16] Jia K, Gong S. Multi□modal tensor face for simultaneous super□resolution and recognition. In: Proc of Tenth IEEE International Conference on Computer Vision (ICCV'05) Volume 1, volume 2. IEEE, 2005, 1683–1690.

[17] Park S W, Savvides M. Individual kernel tensor-subspaces for robust face recognition: A computationally efficient tensor framework without requiring mode factorization. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 2007, 37(5):1156–1166.

[18] 孙玲芳, 李烁朋. 基于 K-means 聚类与张量分解的社会化标签推荐系统研究. 江苏科技大学学报:自然科学版, 2012, (06):597﹣601.

[19] 陈庆奎, 王海峰, 那丽春,等. 图形处理器通用计算的研究综述. 黑龙江大学自然科学学报, 2012, 029(005):672-679.

[20] Hitchcock F L. Multiple Invariants and Generalized Rank of a P-Way Matrix or Tensor. Journal of Mathematics and Physics, 1928, 7(1□4):39–79.

[21] Tucker L R. Some mathematical notes on three□mode factor analysis. Psychometrika, 1966, 31(3):279–311.

[22] Appellof C, Davidson E. Three□dimensional rank annihilation for multi□component determinations. Analytica Chimica Acta, 1983, 146:9–14.

[23] Pausak S, Pines A, Waugh J. Carbon-13 chemical shielding tensors in single□crystal durene. The Journal of Chemical Physics, 1973, 59(2):591–595.

[24] Herzfeld J, Griffin R, Haberkorn R. 31P chemical-shift tensors in barium diethyl phosphate and urea-phosphoric acid: model compounds for phospholipid head□group studies. Biochemistry, 1978, 17(14):2711–2718.

[25] Arduengo III A J, Dixon D A, Kumashiro K K, et al. Chemical shielding tensor

中国知网 https://www.cnki.net

of a carbene. Journal of the American Chemical Society, 1994, 116(14):6361–6367.

[26] Lim L H, Comon P. Multiarray signal processing: Tensor decomposition meets compressed sensing. Comptes Rendus Mecanique, 2010, 338(6):311–320.

[27] Zhang M, Ding C. Robust tucker tensor decomposition for effective image representation. In: Proc of Proceedings of the IEEE International Conference on Computer Vision. 2013, 2448–2455.

[28] Cai Y, Zhang M, Luo D, et al. Low-order tensor decompositions for social tagging recommendation. In: Proc of Proceedings of the fourth ACM international conference on Web search and data mining. 2011, 695–704.

[29] Wang Y, Huang T Z. A tensor-based low-rank model for single-image rain streaks removal. IEEE Access, 2019, 7:83437–83448.

[30] Papalexakis E E, Faloutsos C, Sidiropoulos N D. Parcube: Sparse parallelizable tensor decompositions. In: Proc of Joint European Conference on Machine Learning and Knowledge Discovery in Databases. Springer, 2012, 521–536.

[31] Sidiropoulos N D, Papalexakis E E, Faloutsos C. Parallel randomly compressed cubes: A scalable distributed architecture for big tensor decomposition. IEEE Signal Processing Magazine, 2014, 31(5):57–70.

[32] Kolda T G, Bader B W. MATLAB Tensor Toolbox. Technical report, Sandia National Laboratories, 2006.

[33] Solomonik E, Matthews D, Hammond J, et al. Cyclops tensor framework: Reducing communication and eliminating load imbalance in massively parallel contractions. In: Proc of 2013 IEEE 27th International Symposium on Parallel and Distributed Processing. IEEE, 2013, 813–824.

[34] Kang U, Papalexakis E, Harpale A, et al. Gigatensor: scaling tensor analysis up by 100 times□algorithms and discoveries. In: Proc of Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining. 2012, 316–324.

[35] Smith S, Ravindran N, Sidiropoulos N D, et al. SPLATT: Efficient and parallel sparse tensor-matrix multiplication. In: Proc of 2015 IEEE International Parallel and Distributed Processing Symposium. IEEE, 2015, 61–70.

[36] Yang W, Li K, Li K. A pipeline computing method of SpTV for three-order tensors on CPU and GPU. ACM Transactions on Knowledge Discovery from Data (TKDD), 2019, 13(6):1–27.

[37] Choi J, Liu X, Smith S, et al. Blocking optimization techniques for sparse tensor

中国知网 https://www.cnki.net

computation. In:Proc of 2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, 2018, 568–577.

[38] Hayashi K, Ballard G, Jiang Y, et al. Shared-memory parallelization of MTTKRP for dense tensors. In: Proc of Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. 2018, 393–394.

[39] Choi J H, Vishwanathan S V N. DFacTo: Distributed factorization of tensors[J]. arXiv preprint arXiv:1406.4519, 2014.

[40] 廖志芳, 李玲, 刘丽敏,等. 三部图张量分解标签推荐算法. 计算机学报, 2012, 35(12):2625–2632.

[41] 邹本友, 李翠平, 谭力文,等. 基于用户信任和张量分解的社会网络推荐. 软件学报, 2014, 25(12):2852–2864.

[42] 李贵, 王爽, 李征宇,等. 基于张量分解的个性化标签推荐算法. 计算机科学, 2015, 42(2):267-273.

[43] 李喆康. 基于张量分解的交通流数据补全方法研究. 交通治理与空间重塑——2020 年中国城市交通规划年会论文集, 2020, 8.

[44] 张天骐, 喻盛琪, 张天,等. 基于张量分解和多项式库搜索的多天线 NPLC-DS-CDMA 伪码序列估计. 电子与信息学报, 2020, 42(10):2429-2436.

[45] 张德干, 范洪瑞, 龚倡乐,等. 一种基于张量的车辆交通数据缺失估计新方法. 计算机科学, 47(6A):505–511.

[46] 郭伟, 马超, 迪里达尔•库尔班. 基于张量的智能电网大数据降维研究. 信息技术, 2020(9).

[47] 陈国良, 孙广中, 徐云,等. 并行计算的一体化研究现状与发展趋势. 科学通报, 2009, 54(8):1043–1049.

[48] Best C C , Guide P. NVIDIA CUDA Best Practices Guide. 2010.

[49] Kolda T G, Bader B W. Tensor decompositions and applications. SIAM review, 2009, 51(3):455–500.

[50] Smith S, Karypis G. Tensor-matrix products with a compressed sparse tensor. In: Proc of Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms. 2015, 1–7.

[51] Li J, Choi J, Perros I, et al. Model-driven sparse CP decomposition for higher□order tensors. In: Proc of 2017 IEEE international parallel and distributed processing symposium (IPDPS). IEEE, 2017, 1048–1057.

[52] Dagum L, Menon R. OpenMP: an industry standard API for shared-memory programming. IEEE computational science and engineering, 1998, 5(1):46–55.

[53] NELL. http://rtw.ml.cmu.edu/rtw.

中国知网 https://www.cnki.net

[54] MovieLens. https://movielens.org.

[55] Last.fm. http://www.last.fm.

[56] Li J, Ma Y, Vuduc R. ParTI! : A Parallel Tensor Infrastructure for multicore CPUs and GPUs, 2018-Oct. http://parti-project.org. Last updated: Jan 2020.

[57] Nisa I, Li J, Sukumaran-Rajam A, et al. Load-balanced sparse MTTKRP on GPUs. In: Proc of 2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, 2019, 123–133.

中国知网 https://www.cnki.net

附录 A 攻读学位期间发表的学术论文与获得的成果

[1] Hu R, Yang W, Zhou X, et al. Performance Analysis and Optimization for MTTKRP of Sparse Tensor on CPU and GPU. 2020 IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems (HPCC/SmartCity/DSS). 2020: 545–550.

中国知网 https://www.cnki.net

附录 B 攻读学位期间参加的科研项目

[1] 国防装备预先研究项目：**********。

中国知网 https://www.cnki.net

致　谢

两年的硕士生活如白驹过隙，回首两年的在校生活，这一切都是我人生中非常重要和难忘的经历。这一切离不开老师们的谆谆教诲，也离不开家人朋友的帮助支持。这两年的学习不仅使我的眼界得到开阔，还锻炼了我专业技能和解决问题的能力。谨以此对老师、家人与朋友致以最真挚的感谢。

特别感谢我的导师周旭副教授，非常感谢她在学习、工作和生活上对我的指导和关心。周旭老师是一位非常敬业负责的老师，在我刚入校就参考我的意愿，对未来两年做了规划。两年来，周旭老师积极帮助我解决各种学术问题，告诉我以积极的心态面对生活。

特别感谢我的指导老师阳王东教授。阳王东老师是一个非常敬业且科研水平突出的老师，在我硕士学习期间一直影响着我。在阳老师的指点下我认真踏实做好科研工作，对许多专业知识有了更深入的了解。遇到困惑之处，老师的耐心解惑让我受益匪浅，为我指明下一步的思考方向。在论文撰写的过程中，阳老师给我提了很多宝贵的意见。在设计实验时，阳老师不厌其烦与我讨论，提出方案的可行性。

特别感谢李肯立教授，李老师是我来学校最早接触的老师，时常关心我们是否遇到困难，培养我们良好的作息。李肯立老师所展现出的渊博知识和严谨的治学态度深深感染着我，是我终身学习的榜样。

感谢实验室的所有同窗们。在我对自己不熟悉的领域有困惑时，你们总是第一时间为我答疑解惑。难忘你们在两年对我的关心和帮助，你们勤奋认真的学习态度影响着我。愿大家在未来的日子里更上一层楼。同时也感谢寝室同学的陪伴，怀念那些畅所欲言的日子，为我的科研生活增添了许多光彩。

感谢我亲爱的父母和家人，在我硕士期间无微不至的照顾，使我能够安心学习，你们是我永远可以停靠的港湾。

最后向在百忙之中抽出时间参与我的论文评阅与答辩的老师们致以最衷心的感谢与最诚挚的祝福！

胡蓉

2021 年 5 月

中国知网 https://www.cnki.net